Stratified Software Development

A Layer Calculus for Agent-Authored Systems

Section titled “A Layer Calculus for Agent-Authored Systems”

Formal Technical Report · November 2026

Abstract

Section titled “Abstract”

Autonomous coding agents fail in three recurring ways. They leap from a requirement to code and invent the missing design; they lose the rationale for load-bearing choices and later weaken tests to make builds pass; and they leave humans with no trustworthy intermediate view of what the system does. This report presents a layer calculus that makes software development a sequence of verified behavior inclusions. A layer records intents, their support, design decisions, state machines, a decomposition into child contracts, and evidence. One relation — behavior inclusion under an abstraction map — serves for intent satisfaction, contract refinement, connector decomposition, and code realization. Stratification turns support evaluation into a forward recurrence, sibling verification into a substitution lemma, and version evolution into proof reuse at unchanged interfaces. The central theorem states that a valid stratified development refines the admitted root intents, conditional on explicit executable-realization and profile-admissibility premises. The report develops the layer definition, the soundness proof, the cutoff theorem, a five-gate agent protocol, a four-step evolution loop, and a complete durable-acknowledgement example, and it specifies a small mechanization target.

---

1. Introduction

Section titled “1. Introduction”

1.1 The problem

Section titled “1.1 The problem”

An agent is asked to add a feature or change a subsystem. It receives a requirement sentence and produces code. Between the sentence and the code lie decisions the sentence does not contain: data layout, ownership, error semantics, protocol ordering, retry policy, concurrency control, and edge-case handling. When the agent fills that gap in one leap, three failures follow.

Wide abstraction leap. The agent invents the missing design while writing the implementation. Implicit choices become hidden assumptions: a synchronous clock, a lossless channel, an atomic write that is not atomic, a lock that is not held across the operation that needs it. The resulting system can pass unit tests and still violate the requirement under interleaving, failure, or scale.

Rationale erasure and silent test weakening. A source repository records the final code, not the ordered argument for why a structure or protocol was chosen from upstream constraints, nor why the alternatives were rejected. A later agent cannot distinguish a load-bearing invariant from an incidental detail. Faced with a failing test, the cheapest repair is often to delete the assertion or weaken the condition. The build passes; the invariant is gone.

Human inscrutability. There is no trustworthy intermediate view. To know what the system does, a human must read thousands of lines of agent-generated code, reconstruct the design from the implementation, and guess which parts are intentional. Review degrades into rubber-stamping or line-by-line debugging.

1.2 Why post-hoc auditing does not address the problem

Section titled “1.2 Why post-hoc auditing does not address the problem”

An audit that runs after the code exists can detect some violations, but it does not prevent the leap that produced them. A diff algorithm can show what changed, but it cannot show which invariant the change was supposed to preserve. A test suite can encode a requirement,

but a test is evidence about a sample, not a proof of a universal behavioral obligation, and it is exactly the artifact an agent can weaken. The problem is structural: the development process has no mandatory intermediate objects that are both behavioral and checkable.

1.3 The stratified thesis

Section titled “1.3 The stratified thesis”

Development proceeds through a persistent sequence of behavioral layers

$$L_0 \longrightarrow L_1 \longrightarrow \cdots \longrightarrow L_L \longrightarrow \text{executable code.}$$
$$\mathsf{L0} \longrightarrow \mathsf{L1} \longrightarrow \cdots \longrightarrow \mathsf{LL} \longrightarrow \text{executable code.}$$

At each layer kk, the developer records:

- the intents active at layer kk and the support that activates them;
- the design decisions taken at layer kk, with their alternatives and reopening conditions;
- explicit state machines for the components at layer kk;
- a decomposition of each component into child contracts for layer $k+1$k + 1;
- evidence that the layer’s machines satisfy its intents and that its decomposition is sound.

A candidate advances only when the required checks at that layer succeed. Layer LL is the executable layer: each leaf machine is realized by a program under a declared execution semantics. Stringent layer validity includes the operational realization of the wiring and the selected profile’s initialization, input, and progress conditions; the soundness theorem is conditional on them. The result is a chain of behavior inclusions, one layer at a time, where every link is small enough to verify and every link is retained as evidence.

The prevention claim is precise. An implementation that violates a protected, formally expressed obligation cannot be accepted by a sound gate whose complete dependencies are preserved. The gate does not make the agent intelligent; it makes the agent’s obligations explicit, local, ordered, and checkable.

1.4 Contributions

Section titled “1.4 Contributions”

1. A layer calculus in which intent satisfaction, contract refinement, connector decomposition, and code realization are one relation: behavior inclusion.
2. A stratified support recurrence with a separate evidence stratum and a protected–obligation preservation rule, so neither a failed check nor a support edit silently erases the obligation.
3. A stratified soundness theorem whose chain is executable realization, connector monotonicity, child refinement, and inclusion transitivity.
4. A sibling–independence theorem with explicit composition conditions, including interaction ownership and executable closure.
5. A key–locality cutoff theorem: a change that preserves an interface reuses every ancestor proof by substitution.
6. A five–gate development protocol and a four–step evolution loop, including a repair taxonomy that separates semantic counterexamples, failed proof witnesses, and unresolved checker results.
7. A complete durable–acknowledgement example, worked through three layers, with a single–use interface precondition, a visible durability observation, leaf pseudocode, an agent error, its counterexample, and two repair paths.
8. A mechanization target small enough to audit in one sitting.

1.5 Roadmap

Section titled “1.5 Roadmap”

Section 2 fixes the refinement algebra. Section 3 defines layers, support, decisions, validity, and proves stratified soundness. Section 4 details the four verification obligations and the evidence discipline. Section 5 treats decomposition, connectors, and sibling independence. Section 6 gives the development protocol. Section 7 gives versions, keys, cutoff, and evolution. Section 8 works the durable–acknowledgement example. Section 9 specifies the mechanization target. Sections 10—11 cover related work and open problems. The appendices record the finite probes that motivate the counterexamples and give proof details.

---

2. The Refinement Algebra

Section titled “2. The Refinement Algebra”

2.1 Observations, machines, and specifications

Section titled “2.1 Observations, machines, and specifications”

Fix an interface XX. An **observation space** O_XOX is a set of observations of XX. The core instance is the set of finite traces over the visible actions of XX; the liveness instance adds maximal traces under a declared fairness condition; the timed and probabilistic instances replace traces with timed or measure-valued observations. A **specification** at XX is a subset $\Phi \subseteq O_X$Φ ⊆ OX.

A **machine** at XX is a structure

$$M = (S, S_0, \Sigma, \rightarrow, \mathrm{obs}_M)$$
$$\mathsf{M} = (\mathsf{S}, \mathsf{S0}, \Sigma, \rightarrow, \mathsf{obsM})$$

where SS is a state set, $S_0 \subseteq S$S0 ⊆ S the initial states, ΣΣ a partitioned alphabet of inputs, outputs, and internal actions, $\rightarrow \subseteq S \times \Sigma \times S$ →⊆ S × Σ × S the transition relation, and obs_MobsM maps executions to observations. Its **behavior** is

$$\mathrm{beh}(M) = \{\mathrm{obs}_M(\rho) \mid \rho \text{ is an execution of } M\} \subseteq O_X.$$
$$\mathsf{beh}(\mathsf{M}) = \{\mathsf{obsM}(\rho) \mid \rho \text{ is an execution of } \mathsf{M}\} \subseteq \mathsf{OX}.$$

For the trace kernel, $\mathrm{beh}(M)$beh(M) is a trace set and is written $\mathrm{tr}(M)$tr(M).

2.2 Satisfaction, refinement, and abstraction

Section titled “2.2 Satisfaction, refinement, and abstraction”

Satisfaction and **refinement** are inclusion:

$$M \vDash \Phi \iff \mathrm{beh}(M) \subseteq \Phi, \qquad M \preceq A \iff \mathrm{beh}(M) \subseteq \mathrm{beh}(A).$$
$$\mathsf{M} \vDash \Phi \iff \mathsf{beh}(\mathsf{M}) \subseteq \Phi, \qquad \mathsf{M} \preceq \mathsf{A} \iff \mathsf{beh}(\mathsf{M}) \subseteq \mathsf{beh}(\mathsf{A}).$$

With an abstraction map $\alpha: O_{X'} \to O_X$α : OX′ → OX,

$$M \preceq_\alpha A \iff \alpha(\mathrm{beh}(M)) \subseteq \mathrm{beh}(A), \qquad \alpha^{*}\Phi = \{o' \in O_{X'} \mid \alpha(o') \in \Phi\}.$$
$$\mathsf{M} \preceq_{\alpha} \mathsf{A} \iff \alpha(\mathsf{beh}(\mathsf{M})) \subseteq \mathsf{beh}(\mathsf{A}), \qquad \alpha_{*}\Phi = \{o' \in \mathsf{OX}' \mid \alpha(o') \in \Phi\}.$$

Inclusion is reflexive and transitive. The inverse-image operation is functorial by function composition:

$$(\alpha_2 \circ \alpha_1)^{*} = \alpha_1^{*} \circ \alpha_2^{*}, \qquad \mathrm{id}^{*} = \mathrm{id}.$$
$$(\alpha 2 \circ \alpha 1)_{*} = \alpha 1_{*} \circ \alpha 2_{*}, \qquad \mathsf{id}_{*} = \mathsf{id}.$$

2.3 Connectors

Section titled “2.3 Connectors”

A **connector** at a parent cell vv is a map on behavior sets,

$$K_v: \prod_{c \in \mathrm{ch}(v)} P(O_{X_c}) \longrightarrow P(O_{X_v}),$$
$$\mathsf{Kv} : \prod_{\mathsf{c} \in \mathsf{ch(v)}} \mathsf{P(OXc)} \longrightarrow \mathsf{P(OXv)},$$

and it is **admissible** when it is monotone:

$$B_c \subseteq B_c' \text{ for all } c \implies K_\upsilon(\{B_c\}_c) \subseteq K_\upsilon(\{B_c'\}_c).$$
Bc ⊆ Bc′ for all c ⟹ Kv({Bc}c) ⊆ Kv({Bc′}c).

The operational reading is a wiring skeleton W_υWv that routes, synchronizes, interleaves, and hides child actions, with no state of its own that is not visible in the children or in a declared substrate primitive. A **trajectory model** of W_υWv supplies:

- a set $\mathrm{legal}_{W_\upsilon}$legalWv of legal shared trajectories rr;
- for each child cc, a slice $\mathrm{slice}_c(r)$slicec(r) in O_{X_c}OXc;
- a projection $\mathrm{obs}_\upsilon(r)$obsv(r) in O_{X_υ}OXv.

The induced transformer is

$$K_\upsilon(\{B_c\}_c) = \left\{ \mathrm{obs}_\upsilon(r) \;\middle|\; \mathrm{legal}_{W_\upsilon}(r) \wedge \forall c.\, \mathrm{slice}_c(r) \in B_c \right\}.$$
Kv({Bc}c) = {obsv(r) | legalWv(r) ∧ ∀c. slicec(r) ∈ Bc}.

Lemma 1 (Trajectory connectors are monotone). If $B_c \subseteq B_c'$Bc ⊆ Bc′ for every cc, then $K_\upsilon(\{B_c\}_c) \subseteq K_\upsilon(\{B_c'\}_c)$Kv({Bc}c) ⊆ Kv({Bc′}c).

Proof. Take $o \in K_\upsilon(\{B_c\}_c)$o ∈ Kv({Bc}c). There is a legal rr with $\mathrm{obs}_\upsilon(r) = o$obsv(r) = o and $\mathrm{slice}_c(r) \in B_c$slicec(r) ∈ Bc for every cc. Since $B_c \subseteq B_c'$Bc ⊆ Bc′, each slice lies in B_c'Bc′ as well, so $o \in K_\upsilon(\{B_c'\}_c)$o ∈ Kv({Bc′}c). ■■

The trajectory model is the mathematical content of the wiring. Its **operational soundness** obligation is that the implementation of W_υWv produces only observations in the transformer's output. This is where routing, synchronization, initialization, and shared-resource behavior are established. It is a property of the connector, discharged once per wiring pattern, not a per-child obligation.

2.4 The single relation and its five uses

Section titled "2.4 The single relation and its five uses"

Every behavioral obligation in the calculus is an instance of inclusion:

Use	Left side	Right side	Where checked
Satisfaction	$\mathrm{beh}(M_\upsilon)$beh(Mv)	active intent ΦΦ	layer kk
Wiring adequacy	$\mathrm{Beh}(C_\upsilon[Q_c])$Beh(Cv[Qc])	$K_\upsilon(\{\alpha_c(Q_c)\}_c)$Kv({αc(Qc)}c)	layer kk, universally over admissible Q_c Qc
Decomposition	$K_\upsilon(\{B_{\upsilon,c}\}_c)$Kv({Bv,c}c)	$\mathrm{beh}(M_\upsilon)$beh(Mv)	layer kk
Refinement	$\mathrm{beh}(M_c)$beh(Mc)	contract $B_{\upsilon,c}$Bv,c	layer $k+1$k + 1
Realization	$\mathrm{beh}(P_\ell)$beh(Pℓ)	$\mathrm{beh}(M_\ell)$beh(Mℓ)	layer LL

The algebra is small on purpose. The paper's correctness argument is the transitive closure of this table, and the engineering content is the discipline that decides what to put on each side.

2.5 Semantic profiles

Section titled "2.5 Semantic profiles"

A **semantic profile** ΓΓ fixes the observation class and the obligations that accompany it:

- **Safety profile.** Observations are finite visible traces; $M \preceq A$M ≤ A is trace inclusion.
- **Liveness profile.** Observations are maximal traces under a declared fairness condition; refinement preserves maximality and fairness, and a progress obligation accompanies the trace inclusion.

- **Timed and probabilistic profiles.** Observations are timed traces or path measures; refinement uses the corresponding measurable or timed preorder.

Two side conditions belong to every open-interface profile:

- **Input receptiveness.** If a contract permits an input at a related state, the child must accept it. Without this condition, a child can satisfy a safety contract by refusing every request, which is not an admissible replacement.
- **Progress preservation.** For a liveness contract, the child must preserve the required maximal fair executions. Finite-prefix inclusion alone admits a machine that accepts a request and then stops.

The core theorems are stated for the trace profile with these two side conditions. The timed and probabilistic instantiations use the same layer structure and the same schema; only the profile's refinement relation and composition lemma change.

2.6 Boundary discipline

Section titled "2.6 Boundary discipline"

A contract constrains observable behavior at the child interface and the child's duties to the connector. It may specify:

- the inputs the child must accept;
- the outputs it may produce and their order;
- relations between its outputs and shared connector state;
- progress or termination duties;
- the environment assumptions under which those duties hold.

It does not prescribe the child's internal algorithm, data layout, module structure, or implementation strategy. The child is free to choose any implementation whose behavior lies within the contract. This is the boundary rule that makes independent development possible: **the parent owns the interface; the child owns the implementation.**

---

3. Layers and Stratified Development

Section titled "3. Layers and Stratified Development"

3.1 The layer record

Section titled "3.1 The layer record"

A **layer** is a finite record

$$L_k = \big(I_k,\ J_k,\ D_k,\ V_k,\ \chi_k,\ E_k\big).$$
Lk = (Ik, Jk, Dk, Vk, χk, Ek).

The fields have distinct roles.

I_k**Ik — intent records.** A finite set of intents. Each $c \in I_k$c ∈ Ik carries:

- a stable identifier $\mathrm{id}(c)$id(c) and an exact content digest;
- an owner cell $\mathrm{owner}(c) \in V_k$owner(c) ∈ Vk; an interaction-boundary label may annotate the owner for explanation, but the owner is always a cell;
- a predicate $[\![c]\!] \subseteq O_{X_{\mathrm{owner}(c)}}$[[c]] ⊆ OXowner(c);
- an authority tag: rootroot, $\mathrm{protected}$protected, or $\mathrm{delegated}$delegated.

A requirement that has no formal interpretation is recorded as an intent with a pending formalization status; it remains visible and blocks the obligations that depend on it. Ownership is total and unique: every active intent is owned by exactly one cell, so the per-cell required sets cover A_kAk.

J_kJk **— support links.** For each $c \in I_k$c ∈ Ik, a finite set $J_k(c)$Jk(c) of support groups. Each group $P \in J_k(c)$P ∈ Jk(c) is a finite set of references to earlier intents or earlier selected decisions, including contract selections. A group is conjunctive: all of its references must be active. Multiple groups are alternatives: any one suffices. The support link is provenance; when it claims logical entailment rather than an accepted design rationale, it carries a separate proof obligation.

D_kDk **— decision records.** A finite set of design decisions. Each decision $d \in D_k$d ∈ Dk has:

- a question identifier $q(d)$q(d);
- a selected option $o(d)$o(d);
- a finite set $\mathrm{alt}(d)$alt(d) of rejected alternatives, each with an assessment and a reopening condition;
- a finite prerequisite set $\mathrm{prem}(d)$prem(d) of references;
- a rationale $\mathrm{why}(d)$why(d);
- a selection proposition $\mathrm{sel}(q(d), o(d))$sel(q(d), o(d));
- a reopening predicate $\mathrm{reopen}(d)$reopen(d) on the environment or version.

The ledger also carries a finite set N_kNk of **nogoods**: each nogood is a finite set of selection propositions that cannot all be true. A decision is **evaluated independently**: its prerequisites may mention the layer's active intents and earlier layers' selections, but not the selections of the same layer. After the layer's decisions are evaluated, the joint consistency of S_kSk with the nogoods is checked. If two decisions need each other, they are placed at different layers or merged. This restriction is what keeps the recurrence a single pass.

V_kVk **— cells.** A finite set of components. Each cell $v \in V_k$v ∈ Vk owns:

- an interface X_vXv and observation space O_{X_v}OXv;
- a machine M_vMv at X_vXv;
- its active intents $\Phi_k(v) = \bigcap_{c \in A_k(v)} [\![c]\!]$Φk(v) = ∩c∈Ak(v)[[c]], where $A_k(v) \subseteq A_k$Ak(v) ⊆ Ak is the set of active intents owned by v v;
- a semantic context Γ_vΓv (profile, fairness, environment assumptions).

χ_kχk **— decomposition or realization.** For $k < L$k < L, each non-leaf cell $v \in V_k$v ∈ Vk has a decomposition record

$$\chi_k(v) = \big(W_v,\ \mathrm{ch}(v),\ (B_{v,c})_{c \in \mathrm{ch}(v)},\ \alpha_v\big),$$
χk(v) = (Wv, ch(v), (Bv,c)c∈ch(v), αv),

where W_vWv is a wiring skeleton, $\mathrm{ch}(v) \subseteq V_{k+1}$ch(v) ⊆ Vk+1 is a finite set of child cells, $B_{v,c}$Bv,c is the contract assigned to child cc at its interface, and α_vαv is the parent abstraction map. The children of the cells at layer kk partition V_{k+1}Vk+1, and every cell at layer $k < L$k < L has at least one child. A leaf is a cell at layer LL; its decomposition record is a realization record. A cell that is already executable at layer kk is represented by a trivial decomposition into a single child at layer $k+1$k + 1 with the same machine, repeated until layer LL. Declaring a contract is a design decision and is recorded as a selection in D_kDk; the support links of the next layer reference it through S_kSk.

At $k = L$k = L, $\chi_L(\ell)$χL(ℓ) is a program P_ℓPℓ together with an execution semantics for the leaf $\ell\ell$.

E_kEk **— evidence.** A finite map from obligation keys to evidence records. An evidence record carries the exact obligation, the method, the claim established, the result, the scope, and the artifact. Section 4.5 defines the validity condition. The evidence stratum is separate from the intent stratum: an intent's activation does not depend on whether its evidence exists or passes.

3.2 Active intents and the support recurrence

Section titled "3.2 Active intents and the support recurrence"

Let S_mSm be the set of selection propositions of the decisions accepted at layer mm. The **active intents** are computed top-down:

$$A_0 = I_0^{\text{root}},$$
A0 = I0root,

$$A_k = \left\{ c \in I_k \;\middle|\; \exists P \in J_k(c).\, P \subseteq \bigcup_{m=0}^{k-1} \left(A_m \cup S_m \right) \right\} \qquad (1 \le k \le L).$$
Ak = {c ∈ Ik | ∃P ∈ Jk(c). P ⊆ m=0∪k−1(Am ∪ Sm)} (1 ⩽ k ⩽ L).

The recurrence has no fixed point. Support is strictly earlier-layer, so a single forward pass settles every layer. It is deterministic for a fixed candidate and costs the number of intent records, support groups, and premise occurrences.

The per-layer order is:

1. compute A_kAk from the lower layers;
2. resolve the decisions D_kDk against A_kAk and the earlier selections, and check consistency;
3. verify the layer's behavioral obligations;
4. for $k < L$k < L, declare the contracts that feed layer $k+1$k + 1, and record each declaration as a decision selection in D_kDk.

Lemma 2 (Evidence separation). A_kAk is a function of (I, J, D, S)(I, J, D, S) only. It does not mention E_kEk. Consequently, a failed, missing, or pending evidence record leaves every obligation it concerns active.

Proof. The recurrence defining A_kAk refers only to intent records, support groups, and decision selections. The evidence stratum E_kEk does not occur in it. ■■

Lemma 2 is the anti-vacuity condition. If the gate defined the obligation set as "the intents whose evidence passes," a failed requirement would disappear before it was checked, and the remaining set would be vacuously satisfied. The separation makes failure visible and blocking.

Corollary (No vacuous acceptance). A layer with an active, evidenced obligation whose evidence is absent or failing is not valid. The obligation remains in A_kAk and is reported as unsatisfied.

3.3 Decisions, selection, and consistency

Section titled "3.3 Decisions, selection, and consistency"

A decision is a provenance record, not a theorem. Its **provenance validity** is the prerequisite check:

$$\text{prem}(d) \subseteq \bigcup_{m \le k} A_m \cup \bigcup_{m < k} S_m$$
prem(d) ⊆ m⩽k∪ Am ∪ m<k∪ Sm

for a decision at layer kk. Its **behavioral validity** is checked separately: the machine or contract named by the selected option must satisfy the relevant behavioral obligations. The two checks are independent. A decision can be warranted while its chosen machine fails satisfaction; the gate catches the latter. A machine can satisfy its intents while the decision's rationale is stale; the gate catches the former.

The set S_kSk is the set of selection propositions of accepted decisions at layer kk. A decision is accepted when:

1. its prerequisites are available;
2. its selection is consistent with the nogoods introduced at layers $\le k$⩽ k and with the selections of lower layers;
3. it is not marked stale.

The **consistency** condition is

$$\forall n \in \bigcup_{m \le k} N_m.\quad n \notin \bigcup_{m \le k} S_m.$$
∀n ∈ m≤k∪ Nm. n m≤k∪ Sm.

A nogood whose selections span a later layer is not violated until those selections exist; the check at the later layer includes the earlier nogood. Because selection is explicit, there is no disjunctive closure to compute and no nonmonotonic inference to guard. A question with several acceptable options is represented by several decisions or by one decision with alternatives; exactly one selection is active. The rejected alternatives remain in the ledger, addressable by their reopening conditions, without entering the active semantics.

Staleness. If an intent, contract, or decision xx changes, the decisions whose prerequisite closure contains xx are stale:

$$\mathrm{stale}(x) = \{d \mid x \in \mathrm{prem}^*(d)\}.$$
stale(x) = {d | x ∈ prem∗(d)}.

The prerequisite graph is acyclic because prerequisites are strictly earlier-layer or same-layer intents (not same-layer selections), so the reverse-reachable set is computed in linear time. A stale decision blocks publication of every layer in which it is a prerequisite of an active intent. Re-warranting it, or replacing it with a new decision and reopening its alternatives, is the repair.

A **reopening predicate** fires independently of premise changes. It is evaluated against the current environment and version. The predicate keeps rejected alternatives alive as first-class objects: when the condition that motivated the rejection no longer holds, the decision is stale and the alternative is reconsidered.

3.4 Intra-layer validity

Section titled “3.4 Intra-layer validity”

Before the validity conditions, define the executable behavior of a sub-development. For a leaf ℓℓ, $\mathrm{Beh}_{\mathrm{exec}}(\ell) = \mathrm{beh}(P_\ell)$Behexec(ℓ) = beh(Pℓ) under the declared execution semantics. For a decomposed cell vv, let C_vCv be the concrete wiring implementation and let $\mathrm{Beh}_{\mathrm{exec}}(v)$Behexec(v) be the set of observations C_vCv produces when it runs with the executable sub-developments of vv's children under the declared substrate and scheduling semantics. The transformer K_vKv is the abstraction of C_vCv: $\mathrm{Beh}_{\mathrm{exec}}(v)$Behexec(v) is the concrete object, and K_vKv is its specification-level image. Observation maps are typed: obs_cobsc maps a child execution to O_{X_c}OXc, α_c αc maps O_{X_c}OXc to the contract interface, and obs_vobsv maps a parent execution to O_{X_v}OXv; the maps compose through the refinement tree.

L_kLk is **valid** when seven conditions hold.

1. **Support well-formedness and ownership coverage.** Every active intent in A_kAk has a support group whose references are available; the support links are acyclic; and the per-cell ownership sets cover the active set:

   $$\bigcup_{v \in V_k} A_k(v) = A_k,$$
   v∈Vk∪ Ak(v) = Ak,

   with each active intent owned by exactly one cell. An interaction-boundary label is explanatory metadata on the owning cell; it does not create an unchecked owner.
2. **Decision resolution.** Every decision in D_kDk is accepted, and the selections are consistent with all nogoods introduced at layers $\le k$ ⩽ k and with lower layers.
3. **Satisfaction.** For every cell $v \in V_k$v ∈ Vk and every active intent $c \in A_k(v)$c ∈ Ak(v), the evidence E_kEk contains a valid record for

   $$\mathrm{beh}(M_v) \subseteq [\![c]\!].$$
   beh(Mv) ⊆ [[c]].

4. **Decomposition.** For every non-leaf cell $v \in V_k$v ∈ Vk with $\chi_k(v) = (W_v, \mathrm{ch}(v), (B_{v,c})_c, \alpha_v)$χk(v) = (Wv, ch(v), (Bv,c)c, αv), the evidence E_kEk contains a valid record for

$$K_v\left(\{\mathrm{beh}(B_{v,c})\}_{c \in \mathrm{ch}(v)}\right) \subseteq \mathrm{beh}(M_v),$$
$$\text{Kv}(\{\text{beh}(\text{Bv,c})\}\text{c}\in\text{ch}(\text{v})) \subseteq \text{beh}(\text{Mv}),$$

where K_vKv is the transformer induced by W_vWv and α_vαv. Here $\mathrm{beh}(B_{v,c})$beh(Bv,c) is the set of observations the contract permits; a contract is a specification and is read as a behavior set.

5. **Executable realization.** For a decomposed cell vv, the evidence E_kEk contains a **wiring adequacy certificate** for the concrete wiring C_vCv:

$$\forall (Q_c)_{c \in \mathrm{ch}(v)}.\ \left(\forall c.\ Q_c \text{ is an admissible child behavior for } X_c, \Gamma_c\right) \Rightarrow \mathrm{Beh}(C_v[Q_c]) \subseteq K_v\left(\{\alpha_c(Q_c)\}_c\right).$$
$$\forall(\text{Qc})\text{c}\in\text{ch}(\text{v}).\ (\forall\text{c}.\ \text{Qc is an admissible child behavior for Xc,}\Gamma\text{c}) \Rightarrow \text{Beh}(\text{Cv}[\text{Qc}]) \subseteq \text{Kv}(\{\alpha\text{c}(\text{Qc})\}\text{c}).$$

The instance obligation is the specialization $Q_c = \mathrm{Beh}_{\mathrm{exec}}(c)$Qc = Behexec(c):

$$\mathrm{Beh}_{\mathrm{exec}}(v) \subseteq K_v\left(\{\alpha_c(\mathrm{Beh}_{\mathrm{exec}}(c))\}_{c \in \mathrm{ch}(v)}\right),$$
$$\text{Behexec(v)} \subseteq \text{Kv}(\{\alpha\text{c}(\text{Behexec(c)})\}\text{c}\in\text{ch(v)}),$$

derived by application of the universal certificate, not by a separate solver. Admissible here means the child behavior satisfies the interface typing, input-receptiveness, and profile conditions for X_c, Γ_cXc, Γc; condition 6 certifies that the executable child behaviors are admissible. This is the operational-soundness and interaction-completeness condition: the concrete wiring produces only observations admitted by its transformer and its children. For a leaf $\ell \in V_L$ℓ ∈ VL, the evidence E_LEL contains a valid record for

$$\mathrm{Beh}_{\mathrm{exec}}(\ell) = \mathrm{beh}(P_\ell) \subseteq \mathrm{beh}(M_\ell),$$
$$\text{Behexec}(\ell) = \text{beh}(\text{P}\ell) \subseteq \text{beh}(\text{M}\ell),$$

and the leaf's initial state set is nonempty under the declared initialization.

6. **Profile admissibility.** The selected semantic profile's conditions hold:
   - *Initialization.* Every initial state is in the invariant, and the initial state set is nonempty.
   - *Input receptiveness.* If a contract permits an input at a related state, the child accepts it.
   - *Progress.* For a liveness claim only, the required maximal fair executions are preserved under the declared fairness and enabling conditions. These conditions are part of validity, not background assumptions. They block the empty-behavior and request-refusal witnesses.
7. **Protected obligation preservation.** Every protected obligation active in the previous version remains active with the same semantic content, unless an authorized disposition records its retirement or weakening. Section 3.8 states the rule.

Conditions 5 and 6 are the executable premises that the soundness theorem consumes. Condition 7 is the anti-erasure premise that the publication gate consumes.

3.5 Inter-layer refinement

Section titled "3.5 Inter-layer refinement"

For $k < L$k < L, the inter-layer operator is

$$L_k \Rrightarrow L_{k+1} \iff \forall v \in V_k.\ \forall c \in \mathrm{ch}(v).\ E_{k+1} \text{ contains a valid record for } \mathrm{beh}(M_c) \subseteq \mathrm{beh}(B_{v,c}) \text{ under } \alpha_c.$$
$$\text{Lk} \Rrightarrow \text{Lk+1} \iff \forall \text{v} \in \text{Vk}.\ \forall \text{c} \in \text{ch(v)}.\ \text{Ek+1 contains a valid record for beh(Mc)} \subseteq \text{beh(Bv,c) under } \alpha\text{c}.$$

The operator compares a child implementation with the contract its parent declared. It does not compare the new child with the old child; the contract is the complete interface. This is the ownership discipline in action. The operator's conclusion is about the child machine;

condition 5 carries it to the executable sub-development.

A **stratified development** of depth LL is a sequence $(L_0, \dots, L_L)$(L0, … , LL) satisfying $L_k \Rrightarrow L_{k+1}$Lk ⇛ Lk+1 for every $k < L$k < L. It is **valid** when every layer is intra-layer valid and every adjacent pair satisfies the operator. It is **complete** when every layer is resolved (no pending decisions) and every required realization obligation is discharged.

3.6 Stratified soundness

Section titled “3.6 Stratified soundness”

Theorem 1 (Stratified soundness). Let $(L_0, \dots, L_L)$(L0, … , LL) be a stratified development. Suppose:

- (A1) every layer is intra-layer valid (conditions 1–7 of §3.4);
- (A2) $L_k \Rrightarrow L_{k+1}$Lk ⇛ Lk+1 for every $k < L$k < L;
- (A3) the observation maps are typed and compose through the refinement tree.

Then for every kk and every cell $v \in V_k$v ∈ Vk, $\mathrm{Beh}_{\mathrm{exec}}(v) \subseteq \mathrm{beh}(M_v)$Behexec(v) ⊆ beh(Mv), and for every active intent $c \in A_k(v)$c ∈ Ak(v), $\mathrm{Beh}_{\mathrm{exec}}(v) \subseteq [\![c]\!]$Behexec(v) ⊆ ⟦c⟧. In particular,

$$\mathrm{Beh}_{\mathrm{exec}}(v_0) \subseteq \Phi_0(v_0) = \bigcap_{c \in A_0(v_0)} [\![c]\!],$$

Behexec(v0) ⊆ Φ0(v0) = c∈A0(v0)∩⟦c⟧,

the semantic conjunction at the identified root v_0v0, not the syntactic set A_0A0.

Proof. Induct on $L - k$L − k.

Base $k = L$k = L. Condition 5 gives $\mathrm{Beh}_{\mathrm{exec}}(\ell) = \mathrm{beh}(P_\ell) \subseteq \mathrm{beh}(M_\ell)$Behexec(ℓ) = beh(Pℓ) ⊆ beh(Mℓ) and the nonempty initial state condition. Condition 3 gives $\mathrm{beh}(M_\ell) \subseteq [\![c]\!]$beh(Mℓ) ⊆ ⟦c⟧ for every active intent cc assigned to ℓℓ.

Step. Assume the claim for every child $c \in \mathrm{ch}(v) \subseteq V_{k+1}$c ∈ ch(v) ⊆ Vk+1, and write $B_c = \mathrm{Beh}_{\mathrm{exec}}(c)$Bc = Behexec(c). Let $B_{v,c}$Bv,c be the contract declared by vv for cc. Condition 5 gives the universal wiring-adequacy certificate; instantiating it at $Q_c = B_c$Qc = Bc, which is admissible by condition 6, gives the operational inclusion

$$\mathrm{Beh}_{\mathrm{exec}}(v) \subseteq K_v\big(\{\alpha_c(B_c)\}_c\big). \qquad (1)$$

Behexec(v) ⊆ Kv({αc(Bc)}c). (1)

The induction hypothesis gives $B_c \subseteq \mathrm{beh}(M_c)$Bc ⊆ beh(Mc), hence $\alpha_c(B_c) \subseteq \alpha_c(\mathrm{beh}(M_c))$αc(Bc) ⊆ αc(beh(Mc)). Monotonicity of K_vKv (Lemma 1) gives

$$K_v\big(\{\alpha_c(B_c)\}_c\big) \subseteq K_v\big(\{\alpha_c(\mathrm{beh}(M_c))\}_c\big). \qquad (2)$$

Kv({αc(Bc)}c) ⊆ Kv({αc(beh(Mc))}c). (2)

The inter-layer operator (A2) gives $\alpha_c(\mathrm{beh}(M_c)) \subseteq \mathrm{beh}(B_{v,c})$αc(beh(Mc)) ⊆ beh(Bv,c). Monotonicity gives

$$K_v\big(\{\alpha_c(\mathrm{beh}(M_c))\}_c\big) \subseteq K_v\big(\{\mathrm{beh}(B_{v,c})\}_c\big). \qquad (3)$$

Kv({αc(beh(Mc))}c) ⊆ Kv({beh(Bv,c)}c). (3)

Condition 4 gives

$$K_v\big(\{\mathrm{beh}(B_{v,c})\}_c\big) \subseteq \mathrm{beh}(M_v). \qquad (4)$$
Kv({beh(Bv,c)}c) ⊆ beh(Mv). (4)

Chaining (1)–(4) yields $\mathrm{Beh}_{\mathrm{exec}}(v) \subseteq \mathrm{beh}(M_v)$Behexec(v) ⊆ beh(Mv). Condition 3 gives $\mathrm{beh}(M_v) \subseteq [\![c]\!]$beh(Mv) ⊆ ⟦c⟧ for every active intent cc assigned to vv. Condition 6 supplies the profile's initialization, input, and progress conditions, so no step uses an empty behavior or a refused input to satisfy an obligation. The root case $k = 0$k = 0 gives $\mathrm{Beh}_{\mathrm{exec}}(v_0) \subseteq \Phi_0(v_0)$Behexec(v0) ⊆ Φ0(v0). ■■

The proof is the whole spine. It uses executable realization, monotonicity, child refinement, decomposition, and transitivity. The stratification enters in the induction: every child's proof is a separate premise, and the parent's decomposition is checked against contracts rather than against the concrete product of child states. The operational counterexample — identity KK, child $\{ok\}$ok, parent $\{ok\}$ok, actual wiring $\{bad\}$bad — is excluded by condition 5, because $\{bad\} \subseteq \{ok\}$bad ⊆ ok is false. The empty-initial-state counterexample is excluded by condition 6.

3.7 Layer soundness and inspection

Section titled "3.7 Layer soundness and inspection"

Corollary 1 (Inspection is sound). For every kk and $v \in V_k$v ∈ Vk, $\mathrm{Beh}_{\mathrm{exec}}(v) \subseteq \mathrm{beh}(M_v) \subseteq \Phi_k(v)$Behexec(v) ⊆ beh(Mv) ⊆ Φk(v). The inspection direction is: every executable behavior maps under the abstraction to a behavior permitted by the layer's machines, and every universal obligation displayed at layer kk transfers to the code. The converse does not hold: a layer may deliberately admit abstract behavior that the current implementation does not realize. For example, $\mathrm{beh}(\mathrm{code}) = \{a\}$beh(code) = {a} and $\mathrm{beh}(\mathrm{model}) = \{a, b\}$ beh(model) = {a, b} satisfy refinement, yet bb is abstractly permitted and concretely impossible. The layer is an abstraction, not a claim of behavioral equivalence.

The **inspection view** of layer kk contains:

- the active intents A_kAk, with their support links and authority tags;
- the decision ledger D_kDk, with questions, selections, rejected alternatives, and reopening conditions;
- the cells V_kVk and their machines M_vMv;
- the incoming contracts $B_{\mathrm{par}(v),\,v}$Bpar(v),v that each cell must refine;
- the outgoing decomposition $\chi_k(v)$χk(v) that each non-leaf cell declares;
- the evidence status E_kEk: pass, fail, unresolved, pending, missing, and the scope of each method.

A human inspecting layer kk sees what the system does, what it must do, why it is built this way, and that the view is a sound abstraction. They read code only to audit a specific leaf whose contract they already understand.

3.8 Protected obligation preservation

Section titled "3.8 Protected obligation preservation"

Define the **effective protected inventory** of a version GG:

$$\mathrm{Prot}(G) = \big\{(\mathrm{id}(c),\ [\![c]\!],\ \mathrm{owner}(c),\ \mathrm{scope}(c)) : c \text{ is active in } G \text{ and } \mathrm{authority}(c) \in \{\mathrm{root}, \mathrm{protected}\}\big\}.$$
Prot(G) = {(id(c), ⟦c⟧, owner(c), scope(c)) : c is active in G and authority(c) ∈ {root, protected}}.

A candidate G'G′ **preserves** GG when for every $p \in \mathrm{Prot}(G)$p ∈ Prot(G), either pp is active in G'G′ with the same semantic content, or an authorized disposition $d \in A_{G'}$d ∈ AG′ records pp, the authorization, the reason, and the affected decisions. A support edit, ownership change, selection change, or interpretation change that removes pp without such a disposition is rejected by validity condition 7. Ownership is total, so $\mathrm{Prot}(G)$Prot(G) and the per-cell satisfaction sets range over the same active obligations; a protected interaction intent is assigned to the cell that owns the interaction boundary and is checked there.

The support-only witness: a protected intent cc at layer 1 has $J_1(c) = \{\{s\}\}$J1(c) = {{s}} and ss is accepted. The candidate changes only $J_1(c)$J1(c) to $\varnothing\varnothing$. The identifier, predicate, authority tag, and selection are unchanged, but A_1A1 loses cc, so its satisfaction evidence is no longer requested. Condition 7 rejects the candidate; only an authorized disposition can retire cc. Evidence failure does not deactivate an intent (Lemma 2), and a support or ownership edit does not deactivate a protected intent without a disposition (condition 7).

---

4. Verification Obligations

Section titled "4. Verification Obligations"

4.1 Satisfaction

Section titled "4.1 Satisfaction"

For a finite machine M_vMv and a specification $\Phi\Phi$, satisfaction is checked by exhaustive state-space exploration or by a certified invariant. For an infinite machine, it is checked by a theorem, a certified abstraction, or a bounded argument whose scope is recorded. The obligation is exactly

$$\mathrm{beh}(M_v) \subseteq \Phi.$$
beh(Mv) ⊆ Φ.

A test is not, by itself, evidence for this universal inclusion. It is evidence for a bounded claim: no violation in the tested sample, or no violation within a bound. The evidence record must state the claim; the gate checks that the claim covers the obligation. A universal obligation is not discharged by a sampled test. A bounded intent can be discharged by a bounded check, provided the intent is stated with the same bound.

4.2 Decomposition and connector admissibility

Section titled "4.2 Decomposition and connector admissibility"

The decomposition obligation is

$$K_v\left(\{\mathrm{beh}(B_{v,c})\}_c\right) \subseteq \mathrm{beh}(M_v).$$
Kv({beh(Bv,c)}c) ⊆ beh(Mv).

It is checked at the parent layer, against the abstract contracts, not against the concrete children. The connector's admissibility has four parts:

1. **Monotonicity.** Lemma 1 for the trajectory model; definitional for any transformer presented as a monotone map.
2. **Operational soundness.** The concrete wiring's observations are contained in the transformer's output. Routing, synchronization, initialization, and shared-resource behavior are covered by the trajectory model.
3. **Interaction ownership.** Every inter-child synchronization action and every piece of shared state is owned by the wiring or by a declared substrate service. Children have pairwise disjoint private state. A stateful coordinator is a child cell with its own machine and eventual realization, not an unimplemented oracle inside the connector.
4. **Observation preservation under reduction.** When concrete interleavings are replaced by the transformer's compositional abstraction, the reduction preserves the observations of the profile. Transaction reduction and linearizability are proof methods for this condition when they apply; they are not mandatory ceremony for sequential components.

The wiring-adequacy certificate (parts 1–2) is what the soundness proof consumes; it is validity condition 5, and profile admissibility is condition 6. A layer without them is not valid, and Theorem 1 does not apply. The last two parts are what make the proof applicable to a real concurrent implementation. A connector that fails part 3 or 4 is outside the calculus until the missing owner or the reduction proof is supplied.

4.3 Child refinement

Section titled "4.3 Child refinement"

A child refinement obligation is

$$\mathrm{beh}(M_c) \subseteq_\alpha \mathrm{beh}(B_{v,c}).$$
beh(Mc) ⊆α beh(Bv,c).

For finite trace inclusion, an **inductive simulation witness** establishes it:

- an invariant $I_c \subseteq S_{M_c}$Ic ⊆ SMc;
- a relation $R_c \subseteq S_{M_c} \times S_{B_{v,c}}$Rc ⊆ SMc × SBv,c;
- an abstraction map λ_cλc from child actions to contract actions or silent steps.

The conditions are:

1. **Initiation.** Every initial child state is in I_cIc and is related to an initial contract state.
2. **Consecution.** I_cIc is closed under the child's transitions.
3. **Step simulation.** For $(s,t) \in R_c$(s, t) ∈ Rc and $s \xrightarrow{\sigma} s'$s σ

   ---

   s′: if $\lambda_c(\sigma) = \epsilon$λc(σ) = ϵ, then $(s',t) \in R_c$(s′, t) ∈ Rc; otherwise there is t't′ with $t \xrightarrow{\lambda_c(\sigma)} t'$t λc(σ)

   ---

   t′ and $(s',t') \in R_c$(s′, t′) ∈ Rc.
4. **Input receptiveness.** Every contract input enabled at tt is enabled by some child input in λ_c^{-1}λc−1 at ss.
5. **Silent convergence.** For a liveness profile, a well-founded rank strictly decreases on silent steps, preventing infinite internal stuttering.

Lemma 3 (Finite-trace simulation). Conditions 1–3 imply $\lambda_c(\mathrm{tr}(M_c)) \subseteq \mathrm{tr}(B_{v,c})$λc(tr(Mc)) ⊆ tr(Bv,c).

Proof. Induct on the length of a child execution. Initiation relates the initial states. Consecution keeps the invariant. A silent step keeps the contract state; a visible step appends the matched contract transition. Every finite prefix therefore has a matching contract execution. ■ ■

For state-labeled properties, related states agree on the abstract valuations and silent steps preserve them. For liveness, the witness additionally preserves maximality and fairness, and validity condition 6 supplies the progress rank. For open inputs, simulation condition 4 blocks the trivial refusal strategy.

4.4 Realization

Section titled "4.4 Realization"

At layer LL, each leaf has a program P_ℓPℓ and an execution semantics. The obligation is

$$\mathrm{beh}(P_\ell) \subseteq \mathrm{beh}(M_\ell)$$
beh(Pℓ) ⊆ beh(Mℓ)

under the declared semantics and environment assumptions. The leaf machine's transitions are deterministic conditional on their explicit inputs; residual environment choices are explicit inputs or explicit semantic nondeterminism. The substrate contract names what the runtime guarantees: an atomic write, a flush completion, a message delivery, a clock granularity. The realization obligation is where those guarantees enter the proof. For a decomposed cell, realization is the universal wiring-adequacy certificate of §3.4: $\forall Q_c.\ \mathrm{Beh}(C_v[Q_c]) \subseteq K_v(\{\alpha_c(Q_c)\}_c)$∀Qc. Beh(Cv[Qc]) ⊆ Kv({αc(Qc)}c) over admissible child behaviors; the instance $\mathrm{Beh}_{\mathrm{exec}}(v) \subseteq K_v(\{\alpha_c(\mathrm{Beh}_{\mathrm{exec}}(c))\}_c)$Behexec(v) ⊆ Kv({αc(Behexec(c))}c) follows by application. For a leaf, it is the program-to-machine inclusion above.

4.5 Evidence discipline

Section titled “4.5 Evidence discipline”

An evidence record for an obligation oo is a tuple

$$e = (o,\ \text{method},\ \text{claim},\ \text{result},\ \text{scope},\ \text{artifact}).$$
e = (o, method, claim, result, scope, artifact).

It is **valid** for oo when:

1. claimclaim is at least as strong as oo;
2. methodmethod is sound for claimclaim under scopescope;
3. resultresult is pass.

The result field is one of $\{\text{pass}, \text{fail}, \text{unresolved}, \text{pending}\}${pass, fail, unresolved, pending}. Only `pass` discharges an obligation; `unresolved` and `pending` are not passes and cannot enter the Verified outcome.

The method determines the claim:

Method	Claim it can establish	What it cannot establish
Proof derivation	the exact obligation	nothing beyond the logic’s soundness
Exhaustive finite model check	the obligation for the modeled machine	adequacy of the model to the code
Bounded model check	the obligation up to the bound	the unbounded obligation
Property-based test	no violation in the sampled domain	a universal inclusion
Runtime monitor	no violation in the observed run	future or unobserved behavior
Translation validation	the specific program-to-machine relation	unrelated properties

The scope records the observation profile, machine class, environment assumptions, and any bound. A result is meaningful only within its scope.

4.6 Evidence separation and the three-gate structure

Section titled “4.6 Evidence separation and the three-gate structure”

Every layer has three independent gates:

- **Provenance gate.** Every decision is accepted, its prerequisites hold, its selection is consistent, and no load-bearing decision is stale.
- **Behavioral gate.** Every active intent has valid satisfaction evidence; every decomposition has valid evidence; every child has valid refinement evidence; every executable cell has valid realization evidence (wiring adequacy for a non-leaf cell, program-to-machine for a leaf); the profile’s initialization, input, and (where claimed) progress conditions hold.
- **Preservation gate.** Every protected obligation in the previous version’s effective inventory remains active with the same semantic content, or an authorized disposition records its retirement or weakening. A support, ownership, selection, or interpretation edit that deactivates a protected obligation is rejected.

The separation matters. A failing behavioral check does not remove the intent from A_kAk (Lemma 2). A failing provenance check does not silently delete a machine. A support-only edit cannot deactivate a protected obligation without a disposition (condition 7). All failures are reported with their obligations, their evidence status, and the relevant witnesses. The version is not published until all three gates pass.

4.7 What the gate guarantees

Section titled “4.7 What the gate guarantees”

The gate guarantees that the published executable system refines the active root intents under the declared observation profile and substrate semantics. The guarantee is conditional on validity conditions 5 and 6 (executable realization and profile admissibility) and on

the preservation condition 7, and it is relative to:

- the admitted root intents and their authority;
- the semantic profile and the environment assumptions;
- the soundness of the evidence checkers;
- the connector admissibility proofs;
- the declared execution semantics of the leaf programs.

These are the theorem's hypotheses, not disclaimers. The next section makes the composition conditions explicit.

---

5. Decomposition and Interaction

Section titled "5. Decomposition and Interaction"

5.1 Contracts are relational

Section titled "5.1 Contracts are relational"

A contract that constrains each child separately cannot express a correlation between children. The canonical witness is a parent that admits only the diagonal outcomes.

Proposition 1 (Coordinate projections erase correlation). Let the parent admit the outcome set $R = \{(0,0),(1,1)\}$R = {(0, 0), (1, 1)} over outputs (x, y)(x, y). The coordinate projections are $\pi_1(R) = \{0,1\}$π1(R) = {0, 1} and $\pi_2(R) = \{0,1\}$π2(R) = {0, 1}. Two children that independently refine the projections can jointly emit $(0,1) \notin R$(0, 1) ∈/ R.

Proof. $\{0,1\} \times \{0,1\}${0, 1} × {0, 1} contains $(0,1)$(0, 1), which is not in RR. ■■

The remedy is one of two moves: make the contract relational (a relation between child observations and shared connector state), or hoist the correlation into the connector's wiring. The calculus supports both. The point is that the parent layer must own the correlation; it cannot be pushed into independent child proofs.

5.2 Sibling independence

Section titled "5.2 Sibling independence"

Theorem 2 (Sibling independence). Fix a parent cell $v \in V_k$v ∈ Vk with children $\mathrm{ch}(v) \subseteq V_{k+1}$ch(v) ⊆ Vk+1, contracts $(B_{v,c})_c$(Bv,c)c, wiring W_vWv, and parent machine M_vMv. Suppose:

1. **Child refinement.** For every cc, $\mathrm{beh}(M_c) \subseteq_{\alpha_c} \mathrm{beh}(B_{v,c})$beh(Mc) ⊆αc beh(Bv,c), with input receptiveness and progress preservation under the profile.
2. **Connector monotonicity.** The transformer K_vKv induced by W_vWv is monotone.
3. **Decomposition.** $K_v(\{\mathrm{beh}(B_{v,c})\}_c) \subseteq \mathrm{beh}(M_v)$Kv({beh(Bv,c)}c) ⊆ beh(Mv).
4. **Applicability.** The wiring's operational realization is sound for K_vKv; interaction ownership and reduction preservation hold as required by the profile.

Then

$$K_v\big(\{\alpha_c(\mathrm{beh}(M_c))\}_c\big) \subseteq \mathrm{beh}(M_v),$$
Kv({αc(beh(Mc))}c) ⊆ beh(Mv),

without constructing the concrete sibling product $\prod_c S_{M_c}$Πc SMc.

Proof. Apply the abstraction maps to the child refinements, then apply monotonicity to get $K_v(\{\alpha_c(\mathrm{beh}(M_c))\}_c) \subseteq K_v(\{\mathrm{beh}(B_{v,c})\}_c)$Kv({αc(beh(Mc))}c) ⊆ Kv({beh(Bv,c)}c), and compose with the decomposition. ■■

The theorem is the step case of Theorem 1, stated separately because it is the engineering claim. The children are verified against their contracts, never against each other. The cost of the parent check is the abstract boundary, not the concrete product.

5.3 Executable closure

Section titled “5.3 Executable closure”

A connector that performs stateful coordination cannot remain an unimplemented mathematical object. The calculus requires:

- the wiring W_vWv owns only the specified composition mechanism: routing, synchronizing, interleaving, hiding;
- any stateful coordinator is a child cell with its own machine, contract, and eventual realization;
- any custom router, scheduler, or runtime adapter introduced by the implementation receives its own realization obligation or becomes an explicit substrate assumption;
- no action disappears between the mathematical composition and the executable delivery.

This rule closes the gap that otherwise lets an agent write a beautiful abstract connector and never implement it. The coordinator is refined like everything else.

5.4 Shared services

Section titled “5.4 Shared services”

A service used by several branches is declared at the lowest common ancestor uu as an abstract atomic contract O^{abs}Oabs in the ambient context. Clients invoke it atomically without widening the intermediate interfaces. The service is implemented once, as a separate subtree, and proved to refine O^{abs}Oabs: linearizability for safety, a progress property for liveness. This is a factoring rule, not a new judgment. A stateless both–mover service (logging, telemetry) is an admissible connector with trivial state; its certificates establish the required commutation and progress.

5.5 Contract factoring and capacity

Section titled “5.5 Contract factoring and capacity”

A contract boundary can move history into the child and keep the connector small. With mm idempotent resource bits and the invariant “not all mm are held,” a naive joint recognizer needs 2^m2m states. With per–child two–state contracts that emit signed changes only on a genuine bit change, a connector that counts holders needs $m+1$m+1 states. The gain is architectural: the connector’s state is small, and the history lives in the children. The bound is not a universal polynomial claim; the abstract boundary can still contain a product of contract states when the architecture requires it. The point is to choose the boundary so that the product is small.

5.6 Applicability conditions

Section titled “5.6 Applicability conditions”

The sibling–independence theorem is only as strong as its applicability conditions. The paper states them in the main text because they are the load–bearing assumptions:

- **Interaction ownership.** Every shared interaction has an owning boundary; children have disjoint private state.
- **Operational soundness.** The wiring model covers actual routing, synchronization, initialization, and shared–resource behavior.
- **Input and progress.** The profile’s input receptiveness and progress obligations are preserved by the child refinement.
- **Reduction preservation.** When interleavings are reduced, the reduction preserves the observations of the profile.

A connector that fails one of these is outside the calculus. The failure is a proof obligation, not a silent assumption.

---

6. The Development Protocol

Section titled “6. The Development Protocol”

6.1 The five gates

Section titled “6.1 The five gates”

The protocol is the operational reading of validity. An agent may publish a version only when all five gates pass.

G1 — Read the layer (process duty). Before proposing a change at layer kk, the agent reads the layer’s active intents, support links, decisions, cells, incoming contracts, outgoing decomposition, and evidence status. This is a process obligation on the agent; G2—G5 are checked by the gate.

G2 — No abstraction skip. A machine at layer $k+1$k+1 is introduced only as a child of a contract already declared in some $\chi_k(v)$χk(v). The agent cannot write layer $k+1$k+1 code without first fixing the interface it must satisfy. This is the direct block on the wide abstraction leap.

G3 — Evidence required. Acceptance of a cell requires valid satisfaction evidence for every active intent assigned to it and valid executable–realization evidence: for a non–leaf cell, the universal wiring–adequacy certificate of condition 5; for a leaf, the program–to–machine inclusion. A non–leaf cell also requires valid decomposition evidence. Acceptance of a child requires valid refinement evidence. A test is evidence only for the claim it can establish. A weakened test weakens the claim, not the obligation; the universal obligation remains active and unsatisfied.

G4 — No silent specification weakening. Root and protected intents and contracts can be changed only through an authorized revision that creates a new version and reopens the dependent decisions. The agent may propose a revision; it cannot apply one as part of a code change. A support, ownership, selection, or interpretation edit that deactivates a protected obligation counts as a specification weakening and requires an authorized disposition. The authorization record is part of the version.

G5 — Atomic publication. A version is published only when every obligation in the affected dependency cone has valid pass evidence. Partial layers are not publishable. Missing, unresolved, or pending evidence is not a pass, and a failed local verification cannot be hidden by a successful build elsewhere.

6.2 The authoring cycle

Section titled “6.2 The authoring cycle”

The agent’s per–layer cycle is:

1. **Read.** Load the layer’s inspection view and the incoming contracts.
2. **Decide.** For each open design question, record a decision with its prerequisites, alternatives, rationale, and reopening condition. A decision that changes a protected item requires an authorized revision.
3. **Model.** Propose the layer’s cells and their machines. Each machine must satisfy the active intents assigned to its cell.
4. **Decompose.** For each non–leaf cell, declare the wiring and the child contracts, and discharge the decomposition and wiring–adequacy obligations.
5. **Assign.** Hand each child contract to the next layer’s development. The child may be developed independently within its contract.
6. **Substitute.** When a child is complete, substitute its refinement proof into the parent’s decomposition. Close the parent only when all children are discharged.

A locally checked design with unresolved descendants remains a **development candidate**. The completed–system gate quantifies over all required cells, decompositions, and executable artifacts.

6.3 What the agent may and may not change

Section titled “6.3 What the agent may and may not change”

Artifact	Agent may	Agent may not
Delegated intent	refine, strengthen, or add a child intent	delete or deactivate it via a support/ownership edit without an authorized revision
Protected intent	propose a revision with rationale and reopening	apply the revision, or deactivate it via a support/ownership edit, without an authorized disposition

Artifact	Agent may	Agent may not
Root intent	implement it	silently weaken, reinterpret, or deactivate it
Contract	refine the child implementation; propose a contract revision	change the contract to make a failing child pass
Decision	add, re-warrant, or reopen with rationale	erase the alternatives or the rationale
Machine	choose any implementation within the contract	declare a contract that the parent decomposition does not support
Evidence	add evidence for the exact obligation	substitute a weaker claim for a stronger obligation

6.4 Test weakening, intent revision, and authority

Section titled “6.4 Test weakening, intent revision, and authority”

The test-weakening failure is blocked at the evidence layer. A test is a decision procedure for a bounded or sampled claim. It cannot discharge a universal inclusion unless it is exhaustive for the relevant finite domain. If the agent deletes an assertion, the remaining test establishes a weaker claim; the universal obligation is still active, and the gate reports it as unsatisfied. The only way to remove the obligation is to change its intent, which requires an authorized revision and reopens the dependent decisions. A support, ownership, selection, or interpretation edit that removes a protected obligation is a removal even when the predicate bytes are unchanged; it requires the same authorized disposition.

The authority tags make this concrete:

- **Root intents** are admitted by the owner at version creation.
- **Protected intents and contracts** require an explicit authorization record to change.
- **Delegated intents** may be refined by the agent within the protected boundary.

The authorization record is not a separate governance layer. It is a field of the version, checked by the same gate.

6.5 A transcript of a caught error

Section titled “6.5 A transcript of a caught error”

1. The requirement says: acknowledge a request only after its result is durable.
2. The agent writes a store that emits `committed` after `write` returns.
3. The refinement check for the store contract returns the trace `append, committed` with no `flush_ok`/`fsync`.
4. The evidence record is fail. The store contract remains active.
5. The agent either adds the flush before `committed` (downward repair at the code layer) or, if the substrate cannot support the required durability, introduces a supported flush protocol and implements it, or obtains an authorized revision of the substrate/requirement premise and reopens the store decision.
6. The version is published only after the new evidence passes and the reopen path is recorded.

The next section gives the full example.

---

7. Persistent Versions and Evolution

Section titled “7. Persistent Versions and Evolution”

7.1 Versions

Section titled “7.1 Versions”

A **version** is an immutable snapshot

$$G_t = \left(L_0^{(t)}, \ \dots, L_L^{(t)}, \ A_t\right),$$
$$\mathrm{Gt} = (\mathrm{L0(t)}, \dots, \mathrm{LL(t)}, \ \mathrm{At}),$$

where A_tAt is the authorization record for protected revisions. The snapshot is identified by the hash of its canonical encoding. Publication is a compare-and-swap on the head pointer: a candidate G_{t+1}Gt+1 is installed only if the head still identifies G_tGt. The persistent store — a content-addressed DAG, a Merkle trie, or a plain immutable map — is an implementation choice. The mathematical content of persistence is the locality of obligation keys.

7.2 Obligation keys

Section titled "7.2 Obligation keys"

Each obligation has a key that contains every semantic input to it and nothing else:

$$\begin{aligned}
\kappa_{\mathrm{Wire}}(v) &= h\big(\mathrm{Wire},\ C_v,\ K_v,\ \alpha_v,\ (\alpha_c)_c,\ \Gamma_v,\ (\Gamma_c)_c\big),\\
\kappa_{\mathrm{Sat}}(v,c) &= h\big(\mathrm{Sat},\ M_v,\ [\![\,c\,]\!],\ \Gamma_v\big),\\
\kappa_{\mathrm{Dec}}(v) &= h\big(\mathrm{Dec},\ W_v,\ (B_{v,c})_c,\ M_v,\ \alpha_v,\ \Gamma_v\big),\\
\kappa_{\mathrm{Ref}}(c) &= h\big(\mathrm{Ref},\ M_c,\ B_{\mathrm{par}(c),c},\ \alpha_c,\ \Gamma_c\big),\\
\kappa_{\mathrm{Impl}}(\ell) &= h\big(\mathrm{Impl},\ P_\ell,\ M_\ell,\ \Gamma_\ell\big).
\end{aligned}$$

κWire(v)κSat(v,c)κDec(v)κRef(c)κImpl(ℓ) = h(Wire, Cv, Kv, αv, (αc)c, Γv, (Γc)c), = h(Sat, Mv, ⟦c⟧, Γv), = h(

The key never contains a storage pointer, a subtree hash, or a child implementation that is not a semantic input. Two equal keys denote the same obligation proposition; collisions are resolved by exact comparison of the encoded arguments.

7.3 Cutoff by proof substitution

Section titled "7.3 Cutoff by proof substitution"

Theorem 3 (Cutoff). Let G_tGt be a valid version. Replace the sub-development rooted at a child $c \in V_{k+1}$c ∈ Vk+1 with a new sub-development. Keep fixed the contract $B_{v,c}$Bv,c, the wiring W_vWv, the parent machine M_vMv, the parent's active intents, the parent abstraction map α_vαv, the child abstraction maps $(\alpha_c)_c$(αc)c, the child contexts $(\Gamma_c)_c$(Γc)c, and every sibling. Suppose the new child sub-development is valid and satisfies

$$\mathrm{beh}(M_c') \subseteq \mathrm{beh}(B_{v,c})$$

beh(Mc′) ⊆ beh(Bv,c)

under the same profile and abstraction map α_cαc. Then every obligation at layers $\leq k$⩽ k whose key does not mention the changed objects has an unchanged key and retains its evidence. In particular:

- $\kappa_{\mathrm{Wire}}(v)$κWire(v), $\kappa_{\mathrm{Dec}}(v)$κDec(v), and $\kappa_{\mathrm{Sat}}(v,c')$κSat(v,c′) are unchanged for every parent intent c'c′;
- every sibling $\kappa_{\mathrm{Ref}}(c')$κRef(c′) is unchanged;
- the changed keys are confined to the replaced sub-development: $\kappa_{\mathrm{Impl}}(\ell)$κImpl(ℓ) for a changed leaf, and $\kappa_{\mathrm{Ref}}(c)$κRef(c) if M_c'Mc′ or α_cαc changed. If the child machine and its contract are fixed, $\kappa_{\mathrm{Ref}}(c)$κRef(c) is unchanged as well.

The universal wiring-adequacy certificate is keyed by $(C_v, K_v, \alpha_v, (\alpha_c)_c, \Gamma_v, (\Gamma_c)_c)$(Cv,Kv,αv,(αc)c,Γv,(Γc)c) and does not mention the child tuple. The instance $\mathrm{Beh}_{\mathrm{exec}}(v) \subseteq K_v(\{\alpha_c(\mathrm{Beh}_{\mathrm{exec}}(c))\}_c)$Behexec(v) ⊆ Kv({αc(Behexec(c))}c) is re-derived by applying the unchanged certificate to the new child behaviors; no ancestor solver is re-run. If a child interface map changes, the parent's wiring-adequacy key changes and the applicable wiring evidence must be re-established; the zero-ancestor-solver cutoff assumes the family $(\alpha_c)_c$(αc)c is fixed. If a wiring cannot establish the universal certificate, its evidence is instance-specific; the cutoff claim does not apply to that ancestor, and the affected operational obligation is revalidated.

Proof. The keys are functions of their listed arguments. The parent keys list M_vMv, W_vWv, $B_{v,c}$Bv,c, α_vαv, Γ_vΓv, and the wiring $(C_v, K_v, \alpha_v, (\alpha_c)_c, \Gamma_v, (\Gamma_c)_c)$(Cv, Kv, αv, (αc)c, Γv, (Γc)c); none lists M_c'Mc′ or the child behavior tuple. The sibling keys list their own children and contracts; none lists M_c'Mc′. The child's own keys may change, including κ_{Impl}κImpl for a changed leaf; $\kappa_{\mathrm{Ref}}(c)$κRef(c) changes only if M_c'Mc′ or α_cαc changes. Equal keys denote the same proposition, and the stored evidence proves that proposition; substituting the new child's refinement proof into the parent's derivation is sound by Theorem 2. ■■

The theorem does not require the new child to refine the old child. $\{0\}${0} and $\{1\}${1} are incomparable implementations of the same boundary $\{0,1\}${0, 1}, and either may replace the other when the boundary is unchanged. The persistent store realizes the cutoff with path copying: the ancestor spine is rewritten, the unmodified subtrees are shared, and no ancestor solver is re-run.

7.4 The evolution loop

Section titled "7.4 The evolution loop"

The evolution function

$$G_{t+1} = \mathrm{evolve}(G_t, \Delta I)$$
Gt+1 = evolve(Gt, ΔI)

returns one of three outcomes: **Verified**, **Conflict**, or **Unresolved**. There is no default-to-pass.

Step 1 — Apply and reactivate. Apply ΔIΔI: a new intent, a new decision, a protected revision with authorization, or a contract change. Recompute the active intents A_kAk top-down. Resolve the affected decisions and check consistency. Mark stale decisions by reverse reachability. If a nogood is fully selected, return **Conflict** with the minimal nogood and its supporting decisions; stop before any machine search.

Step 2 — Locate. Walk the layers from the root downward, checking the affected obligation keys. The result of each check is one of: **pass**, **fail** with a semantic counterexample, **failed proof witness** (for example, an inductive invariant or simulation relation that failed), or **unresolved** (timeout or unknown). Let jj be the first layer at which an affected obligation does not pass. Do not take the Verified path merely because no obligation "fails": publish only when every required obligation in the affected cone has valid pass evidence. A missing, unresolved, or pending obligation is not a pass.

Step 3 — Repair at jj. The check result determines the repair:

- **Semantic counterexample, genuine.** A concrete execution admitted by the failing layer's machine violates an active intent, or a concrete interleaving of its children violates the decomposition. Repair the machine, connector, or contracts at that layer; re-establish its obligations and its refinement to the layer above; propagate upward only along the affected spine.
- **Semantic counterexample, spurious child refinement.** The child's model or abstraction overapproximates: it admits behavior the fixed child implementation cannot realize. Repair the child model or its abstraction map by narrowing the abstraction, and re-establish the lower realization and the upper refinement. Narrowing the parent's incoming contract for a fixed child abstraction makes the inclusion harder and cannot remove the spurious behavior; reserve contract narrowing for the decomposition case below.
- **Semantic counterexample, spurious decomposition.** The parent-side child contract is too permissive and the concrete children satisfy a stronger contract. Narrow the parent-side child contract or repair the connector, and re-establish the decomposition. If the concrete children do not satisfy the stronger contract, the repair belongs to the children, not to the contract.
- **Failed proof witness.** An inductive invariant or simulation relation failed. Repair the witness (strengthen the invariant or relation) and recheck. This is a proof-search outcome, not a semantic counterexample and not a machine or contract edit by itself.
- **Unresolved.** The checker timed out or returned unknown. Return **Unresolved**; do not classify it as genuine or spurious, and do not publish.

For a refinement failure, the set example is decisive: implementation $\{ok\}${ok}, child abstraction $\{ok, bad\}${ok, bad}, parent contract $\{ok\}$ {ok}. The implementation satisfies the contract, but the abstraction admits badbad. No stronger parent contract repairs the fixed child abstraction; narrowing the child abstraction to $\{ok\}${ok} does. A child refinement failure is never repaired by strengthening the parent-side contract alone.

The genuine/spurious test for a semantic decomposition counterexample uses one shared trajectory:

$$\exists r.\ \mathrm{legal}_{W_v}(r)\ \wedge\ \mathrm{obs}_v(r) = \tau\ \wedge\ \forall c.\ \mathrm{realizable}\big(M_c, \mathrm{slice}_c(r)\big).$$

∃r. legalWv(r) ∧ obsv(r) = τ ∧ ∀c. realizable(Mc, slicec(r)).

A separate existential trajectory per child is not sufficient. Two children can each be realizable on their own slice while no common trajectory exists. The shared-bit witness is Proposition 2.

Step 4 — Publish. If every obligation in the affected cone has valid pass evidence and every load-bearing decision is warranted, atomically advance the head from G_tGt to G_{t+1}Gt+1 and record the authorization. Otherwise return **Unresolved** with the failing or unresolved obligations and their witnesses.

7.5 Genuine vs. spurious

Section titled “7.5 Genuine vs. spurious”

Proposition 2 (Separate realizations do not imply a joint realization). There are two child contracts, one requiring a shared bit to be $0$0 and one requiring it to be $1$1, such that each is separately realizable but no common trajectory realizes both.

Proof. Let the shared bit be $b \in \{0,1\}$b ∈ {0,1}. Child 1 is realizable only with $b = 0$b = 0; child 2 is realizable only with $b = 1$b = 1. Each existential check succeeds with a different value of bb. No single rr has $b = 0$b = 0 and $b = 1$b = 1. ■■

The proposition is the reason the repair step uses a joint trajectory. It is also the reason a connector that hides shared state behind per-child contracts is unsound: the correlation is not in either contract.

7.6 Staleness and reopening

Section titled “7.6 Staleness and reopening”

A change to an intent, contract, or decision activates the dependency cone. Every decision in the cone is stale until re-warranted. A reopened alternative becomes selectable again; selecting it is a new decision with its own prerequisites and evidence. The prior selection remains in the history, so the audit trail is not rewritten.

The dependency cone is computed on the reference graph, not on the storage graph. A subtree storage change with an unchanged semantic contract does not activate the cone; a semantic change with an unchanged storage pointer does. This is the practical benefit of separating keys from pointers.

7.7 Costs

Section titled “7.7 Costs”

Operation	Cost	Structural reason
Active intent recurrence	$O(\sum_k (\lvert I_k\rvert + \lvert J_k\rvert + \text{premise occurrences}))$O(Σk(|Ik| + |Jk| + premise occurrences))	Strictly earlier-layer support; one pass
Decision resolution and consistency	Linear in decisions plus nogood occurrences	Explicit selection; post-pass
Satisfaction for a cell	Model-checking cost for M_vMv	Local machine, local intent
Wiring adequacy	A universal proof over the admissible child-tuple class for $(C_v, K_v, \alpha_v, (\alpha_c)_c, \Gamma_v, (\Gamma_c)_c)$(Cv, Kv, αv, (αc)c, Γv, (Γc)c); not a state-pair count	One certificate per wiring; independent of the concrete child implementations
Decomposition	Up to $\lvert S_{M_v}\rvert \cdot \lvert S_{K_v}\rvert \cdot \prod_c \lvert S_{B_{v,c}}\rvert$ |SMv| · |SKv| · Πc |SBv,c| candidate relation pairs; the boundary state count without the parent factor is $\lvert S_{K_v}\rvert \cdot \prod_c \lvert S_{B_{v,c}}\rvert$ |SKv| · Πc|SBv,c|	A simulation relation against M_vMv adds the parent factor. This is a state/relation-size bound, not a running-time bound.
Child refinement	Up to $\lvert S_{M_c}\rvert \cdot \lvert S_{B_{v,c}}\rvert$ |SMc| · |SBv,c| candidate relation pairs	Relation size, not generic running time; transition, alphabet, and property-automaton costs are separate.

Operation	Cost	Structural reason
Cutoff at an unchanged boundary	$O(1)$O(1) key/hash lookup per unchanged obligation	Encoding, hashing, and proof checking retain their own costs.
Persistent update	$O(\text{spine length})$O(spine length) index nodes for a persistent balanced map with bounded branching; otherwise the representation-sensitive update-path sum	Copying a large child vector and updating a balanced map have different costs.

The table is a cost profile, not a complexity-class claim. The change region is the dependency cone up to proved unchanged boundaries. A root-obligation change can reach the whole system; "bounded blast radius" means bounded by that cone, not by a fixed constant. Stratification removes the global fixed point and the concrete sibling product from the verification obligation; it does not make the per-layer model-checking problem cheap. Solver search, transition enumeration, and property-automaton construction are accounted separately.

---

8. End-to-End Example: Durable Acknowledgement

Section titled "8. End-to-End Example: Durable Acknowledgement"

8.1 The requirement and scope

Section titled "8.1 The requirement and scope"

The system instance serves one request. The environment precondition is: at most one `req(id)` is delivered to an instance, and the instance is not reused after `ack` or `fail`. A reusable service needs a reset transition and matching contracts; that variant is outside this example.

The declared safety intent is:

- **Safety.** For every identifier idid, an `ack(id)` output occurs only after the result for idid is durable, witnessed at the store boundary by `flush_ok(id)` before `committed(id)`.

The liveness claim is separate and conditional:

- **Conditional liveness.** If the scheduler is fair, the store responds to every `append(id)` with `flush_ok(id)` or `flush_fail(id)`, and the coordinator is fair, then every `req(id)` is eventually followed by `ack(id)` or `fail(id)`.

The safety proof does not establish conditional liveness. Liveness is discharged only under those premises, which are recorded in the profile of the coordinator and store cells. The leaf runtime is run-to-completion, non-reentrant, with a FIFO event queue: `emit` enqueues an event and returns; the current handler completes before the next event is dispatched. Initialization is coordinator state `idle` and store state `empty`. The reentrant interpretation is outside this example; §8.5 also places the coordinator in `appending` before `append` is emitted, so a reentrant response is not discarded.

8.2 Layer 0: the abstract design

Section titled "8.2 Layer 0: the abstract design"

The root cell v_0v0 has the machine M_0M0:

- states: `idle`, `pending(id)`, `durable(id)`, `acked(id)`;
- transitions:
  - `idle --req(id)--> pending(id)`;
  - `pending(id) --commit_ok(id)--> durable(id)`;
  - `pending(id) --commit_fail(id)--> acked(id)` with output `fail(id)`;
  - `durable(id) --tau--> acked(id)` with output `ack(id)`.

The root has no transition out of `acked(id)`; the instance is single-use, matching the environment precondition. The machine satisfies the safety intent by construction: `ack(id)` is emitted only from `durable(id)`, which is reached only by `commit_ok(id)`. Conditional liveness is a separate obligation under the fairness premises; it is not inferred from this safety construction.

The root decision ledger records why the design separates durability from acknowledgement: the alternative of acknowledging on enqueue was rejected because it treats a volatile queue as durable; the reopening condition is a substrate that provides a durable atomic enqueue.

8.3 Layer 0 decomposition

Section titled “8.3 Layer 0 decomposition”

The root declares two children at layer 1: a coordinator `coord` and a durable store `store`. The wiring W_0W0 routes:

- `req(id)` from the environment to `coord`;
- `append(id)` from `coord` to `store`;
- `committed(id)` and `failed(id)` from `store` to `coord`;
- `ack(id)` and `fail(id)` from `coord` to the environment.

The store’s boundary also exposes `flush_ok(id)` and `flush_fail(id)`; the wiring routes them to the environment observer that witnesses durability. The root’s `commit_ok` is the abstraction of the store’s `flush_ok` followed by `committed`; the abstraction map αα is part of the decomposition record.

The contracts are:

$B_{0,\text{coord}}$B0,coord — inputs `req(id)`, `committed(id)`, `failed(id)`; outputs `append(id)`, `ack(id)`, `fail(id)`. Duties: at most one `req(id)` per instance; no second request after `ack` or `fail`; `append(id)` may follow `req(id)`; `ack(id)` only after `committed(id)`; `fail(id)` only after `failed(id)`. Conditional progress: under fair coordinator scheduling and a store response, every `req(id)` eventually receives `ack(id)` or `fail(id)`.

$B_{0,\text{store}}$B0,store — inputs `append(id)`; observable outputs `flush_ok(id)`, `flush_fail(id)`, `committed(id)`, `failed(id)`. Duties: at most one `append(id)` per instance; `committed(id)` only after `flush_ok(id)`; `flush_ok(id)` is the substrate’s durability event; `failed(id)` only after `flush_fail(id)`. Conditional progress: under the substrate’s fairness, every `append(id)` eventually receives `flush_ok(id)` or `flush_fail(id)` and then `committed(id)` or `failed(id)`.

The contracts are not merely necessary ordering duties. Their trace languages are the prefix closures

$$L_{\text{coord}} = \text{pref}(\text{req}(id)\,\text{append}(id)\,\text{committed}(id)\,\text{ack}(id)) \cup \text{pref}(\text{req}(id)\,\text{append}(id)\,\text{failed}(id)\,\text{fail}(id)),$$
Lcoord = pref(req(id) append(id) committed(id) ack(id)) ∪ pref(req(id) append(id) failed(id) fail(id)),

$$L_{\text{store}} = \text{pref}(\text{append}(id)\,\text{flush_ok}(id)\,\text{committed}(id)) \cup \text{pref}(\text{append}(id)\,\text{flush_fail}(id)\,\text{failed}(id)),$$
Lstore = pref(append(id) flush_ok(id) committed(id)) ∪ pref(append(id) flush_fail(id) failed(id)),

with one identifier per instance. The root machine’s trace language is

$$L_0 = \text{pref}(\text{req}(id)\,\text{commit_ok}(id)\,\text{ack}(id)) \cup \text{pref}(\text{req}(id)\,\text{commit_fail}(id)\,\text{fail}(id)).$$
L0 = pref(req(id) commit_ok(id) ack(id)) ∪ pref(req(id) commit_fail(id) fail(id)).

The prefix closure ends at `ack` or `fail`; a second `ack`, a spontaneous output, or an incompatible continuation is not in L_{coord}Lcoord. The duplicate–ack composition trace `req, append, flush_ok, committed, ack, ack` projects onto the coordinator alphabet as `req, append, committed, ack, ack`, which is not in the prefix closure L_{coord}Lcoord; the composition trace is therefore excluded by the contract.

The store boundary observation alphabet explicitly includes `flush_ok` and `flush_fail`. The proof boundary is not the projection that hides them: hiding `flush_ok` would make the good trace `append, flush_ok, committed` and the bad trace `append, committed` indistinguishable, so the durability duty would not be checkable. The contract keeps the durability observation visible.

The decomposition obligation is

$$K_0\big(\mathrm{beh}(B_{0,\mathrm{coord}}),\ \mathrm{beh}(B_{0,\mathrm{store}})\big) \subseteq \mathrm{beh}(M_0).$$
K0(beh(B0,coord), beh(B0,store)) ⊆ beh(M0).

It holds for the single-use protocol: any composition trace is a one-request trace; its `ack` is preceded by the coordinator's `committed`, which is preceded by the store's `flush_ok`; the root's `commit_ok`/`ack` path matches. The `commit_fail`/`fail` path matches the store's `flush_fail`/`failed` and the coordinator's `fail`. The abstraction maps the composition trace `req, append, flush_ok, committed, ack` to `req, commit_ok, ack` and `req, append, flush_fail, failed, fail` to `req, commit_fail, fail`; the composition's trace language is contained in L_0L0. The two-request trace `req(1), append(1), flush_ok(1), committed(1), ack(1), req(2), ...` is excluded by the interface precondition and the one-request trace language. The duplicate-ack trace `req, append, flush_ok, committed, ack, ack` is excluded by the prefix closure L_{coord}Lcoord. The wiring is stateless; the coordinator is a child cell, not an oracle inside the connector.

8.4 Layer 1: the child machines

Section titled "8.4 Layer 1: the child machines"

Coordinator M_{coord}**Mcoord**: states `idle`, `appending(id)`, `ready(id)`, `done(id)`.

- `idle --req(id)--> appending(id)` with output `append(id)`;
- `appending(id) --committed(id)--> ready(id)`;
- `ready(id) --tau--> done(id)` with output `ack(id)`;
- `appending(id) --failed(id)--> done(id)` with output `fail(id)`.

Its local active intent is "no acknowledgement before a committed result." The refinement $M_{\mathrm{coord}} \preceq B_{0,\mathrm{coord}}$Mcoord ≤ B0,coord is a short simulation: `ack` only from `ready`, which is reached only by `committed`; the one-request clause follows from the absence of a transition out of `done`, and the trace language is exactly L_{coord}Lcoord.

Store M_{store}**Mstore**: states `empty`, `writing(id)`, `flushed(id)`, `committed(id)`, `failed`.

- `empty --append(id)--> writing(id)`;
- `writing(id) --flush_ok(id)--> flushed(id)`;
- `flushed(id) --tau--> committed(id)` with output `committed(id)`;
- `writing(id) --flush_fail(id)--> failed` with output `failed(id)`.
- `committed(id)` has no outgoing transition except a silent stutter with no output.

Its local active intent is "committed implies durable." The refinement $M_{\mathrm{store}} \preceq B_{0,\mathrm{store}}$Mstore ≤ B0,store requires that `committed` is reachable only after `flush_ok`, and that `flush_ok` is the substrate's durability event. The boundary trace is `append, flush_ok, committed` (or `append, flush_fail, failed`); `committed` is emitted only after `flush_ok`, and the trace language is exactly L_{store}Lstore.

The layer-1 decisions record the choice of a write-ahead log over an in-memory queue with periodic snapshots. The premises are the durability requirement and the crash model. The rejected alternative's reopening condition is "a durable atomic append primitive is available from the substrate." The decision is provenance; the behavioral obligation is the refinement of the contracts.

8.5 Layer 2: the executable leaves

Section titled "8.5 Layer 2: the executable leaves"

The layer-1 cells decompose into layer-2 leaves with identity wiring; the layer-2 leaf machine is the layer-1 machine. The programs are:

```
P_store:
  on append(id):
    state := writing
    write(log, id)                # volatile write
    r := fsync(log)               # substrate call
    if r = 0: emit flush_ok(id); state := flushed; emit committed(id); state := committed
    else:     emit flush_fail(id); state := failed; emit failed(id)

P_coord:
  on req(id):       state := appending; emit append(id)
  on committed(id): if state = appending: state := ready; emit ack(id); state := done
  on failed(id):    if state = appending: state := done; emit fail(id)
```

The substrate contract is: `fsync` returns 0 iff the record is durable across the modeled crash; `flush_ok(id)` is emitted only on a 0 return. The realization witness for the store relates the log state to `M_store`'s state: `fsync` success corresponds to the `flush_ok` transition, and the `committed` emission corresponds to `flushed -> committed`. The invariant is "a record is in `flushed` only if the corresponding `fsync` returned 0." The good trace `append, flush_ok, committed` is admitted; the bad trace `append, committed` is not, because no `flush_ok` transition occurred. The coordinator realization is the analogous event-loop simulation; `ack` is emitted on the `ready -> done` transition, after `committed`. Under the declared run-to-completion semantics, the response cannot be delivered while the request handler is still running; the program also enters `appending` before emitting `append`, so a synchronous/reentrant response would find the handler in `appending` and would not be discarded. Under the declared semantics and the stated fairness/enabling premises, the coordinator reaches `done` and emits `ack`; this is the conditional liveness claim, not part of the safety realization.

8.6 The agent's initial mistake

Section titled "8.6 The agent's initial mistake"

The agent writes the store to emit `committed` immediately after `write` returns, before `fsync`. The refinement check $P_{\text{store}} \preceq M_{\text{store}}$Pstore ≤ Mstore fails with the finite trace

$$\text{append}(id),\ \text{committed}(id).$$
append(id), committed(id).

The trace violates the durability duty: `committed` occurs without `flush_ok`. The evidence record is fail, and the store contract remains active.

Two repairs are available.

- **Downward repair at layer 2.** Add the `fsync` before `committed`. The new store program satisfies the same contract. Only the store's implementation key `k_Impl(store_code)` changes. The leaf machine `M_store` and its incoming contract `B_{1,store_code} = beh(M_store)` are fixed, so `k_Ref(store_code)` and the parent `k_Dec(store)` are unchanged. The cutoff theorem reuses the parent decomposition and the coordinator proof.
- **Upward refinement.** Strengthening a contract cannot make an incapable substrate provide durability. The upward path is one of two authorized changes: (i) introduce a supported flush protocol and implement it, with the new contract and evidence; or (ii) revise the substrate or requirement premise with an authorized disposition and reopen the store decision. The example assumes the substrate supports `fsync`; the upward path is for a different substrate and must be authorized and re-verified.

The decision ledger records both paths and the condition under which the upward path is preferred. The audit trail is not rewritten: the original failed attempt, its counterexample, and the chosen repair remain in the version history.

8.7 Inspection at each layer

Section titled "8.7 Inspection at each layer"

Layer	What the human sees	What they do not need to read
0	the safety intent, the conditional liveness premise, the abstract durability state machine, the decision rationale	any implementation
1	the coordinator and store machines, the contracts including the visible `flush_ok` durability observation, the decision ledger, evidence status	the log buffer code
2	the WAL code and its realization evidence, scoped to the substrate contract	unrelated modules

At layer 1, the human can verify the acknowledgement protocol: `ack` only after `committed`, `committed` only after a durable flush, and the progress duties. At layer 2, they audit the leaf whose contract they already understand. The soundness corollary guarantees that the layer−1 view is a sound abstraction of the code.

8.8 What the example demonstrates

Section titled “8.8 What the example demonstrates”

The example exercises all three failure modes:

- the wide leap is blocked because the coordinator and store interfaces must be declared before their code;
- the rationale is preserved in the decision ledger, and the rejected in−memory alternative has a reopening condition;
- the human inspects layer 1 rather than the WAL code.

It also exercises the downward repair and the authorized upward path, and the cutoff theorem.

---

9. Mechanization Target

Section titled “9. Mechanization Target”

The calculus is designed to be mechanized in a small dependent−type development. The target is not a proof of the entire engineering stack; it is a kernel−checked proof of the algebraic spine, with the operational and solver obligations as explicit hypotheses.

9.1 Target signatures

Section titled “9.1 Target signatures”

```
-- Schematic target signatures. Proofs and instance details are omitted;
-- this is a mechanization target, not a checked development.
-- Observations and behavior
structure Machine (Obs : Type) where
  beh : Set Obs

def Refines {Obs} (M A : Machine Obs) : Prop := M.beh ⊆ A.beh

theorem refines_refl (M : Machine Obs) : Refines M M
theorem refines_trans : Refines M A → Refines A B → Refines M B

-- A connector is a monotone transformer
structure Connector (ι Obs : Type) where
  K : (ι → Set Obs) → Set Obs
  monotone : ∀ B B', (∀ i, B i ⊆ B' i) → K B ⊆ K B'

-- A layer: intents, support, decisions, cells, decomposition, evidence
structure Layer where
  Intent : Type
  active : Set Intent
  Cell : Type
  machine : Cell → Machine Obs
  intentOf : Intent → Cell
  spec : Intent → Set Obs
  evidence : Obligation → Evidence

-- Development: a finite sequence with adjacent refinement
structure Development where
  depth : Nat
  layer : Fin (depth + 1) → Layer
  child : (k : Fin depth) → (v : (layer k).Cell) → Set ((layer (k+1)).Cell)
  contract : (k : Fin depth) → (v : (layer k).Cell) → (c : (layer (k+1)).Cell) → Set Obs
  dec : (k : Fin depth) → (v : (layer k).Cell) → Prop
  ref : (k : Fin depth) → (v : (layer k).Cell) → (c : (layer (k+1)).Cell) → Prop
  valid : Prop

theorem stratified_soundness :
  Development.valid D →
  ∀ (k : Fin (D.depth + 1)) (v : (D.layer k).Cell),
    Refines (subDevelopment D k v) ((D.layer k).machine v)

theorem cutoff :
  UnchangedContract D D' c →
  ValidChild D' c →
  ∀ ancestorObligation, ancestorObligation ∈/changedCone c →
    Reusable D D' ancestorObligation

theorem evidence_separation :
  Active D k c → Evidence D k c = fail → Active D k c

theorem projection_counterexample :
  ∃ (R : Set (Bool × Bool)),
    R = {(false,false), (true,true)} ∧
    (false,true) ∈ (Prod.fst '' R) ×ˢ (Prod.snd '' R) ∧
    (false,true) ∈/R
```

The actual development instantiates `Obs` with finite traces, defines `subDevelopment` by structural recursion on the refinement tree, and proves the four theorems. The `Layer` record in the target is a schematic; the paper's six-field record is the instantiation.

9.2 Trusted base

Section titled "9.2 Trusted base"

The kernel checks the algebra. The following remain explicit hypotheses, with their scopes recorded:

- the operational soundness of a connector's wiring against its transformer;
- the soundness of the evidence checker for each method;

- the adequacy of the finite model to the code for a model-checking claim;
- the substrate contract for a leaf realization;
- the input-receptiveness and progress obligations for the chosen profile.

A mechanized proof cannot make these hypotheses true. It can make their use explicit and prevent the algebra from hiding them.

9.3 Size

Section titled "9.3 Size"

The target is approximately 250—350 lines of Lean 4 with no axioms and no admitted proofs beyond the hypotheses. The small size is a design objective: a reader can audit the entire correctness spine in one sitting, and a reviewer can see exactly which premises are assumed.

---

10. Related Work

Section titled "10. Related Work"

Refinement calculi. Back and von Wright's refinement calculus and Abrial's Event-B develop programs by stepwise refinement, with invariants and proof obligations at each step. The layer calculus shares the stepwise discipline but makes the interface between steps a first-class contract and adds a persistent provenance ledger. Event-B's refinement is per-machine; the layer calculus adds the connector and the child-contract decomposition, which is what enables independent child verification.

Temporal specification and refinement mappings. TLA+ and the Abadi—Lamport theory of refinement mappings establish implementation relations between specifications. The layer calculus uses the same inclusion idea, specialized to a layered development with explicit contracts. The connector's monotonicity is the composition law; the layer structure is the engineering discipline that keeps the law applicable.

Interface automata and assume-guarantee. de Alfaro and Henzinger's interface automata formalize open components with input/output variance and compatibility. The child refinement obligation's input receptiveness is the same discipline. Assume-guarantee reasoning is powerful but fragile when assumptions are circular; the layer calculus avoids circularity by making the connector own the interaction and by requiring an executable coordinator rather than a circular assumption.

Design by contract. Meyer's contracts specify preconditions, postconditions, and invariants. The layer calculus extends the idea from a single object boundary to a stratified decomposition: the contract is the interface between layers, and its satisfaction is checked by refinement rather than by runtime assertion alone.

Hierarchical state machines. Harel's statecharts provide nested and orthogonal states. The layer calculus uses ordinary state machines at each layer and a connector for interaction; it does not rely on statechart semantics. The hierarchy is in the development structure, not in a single expanded automaton.

Counterexample-guided abstraction refinement. Clarke et al.'s CEGAR loop uses counterexamples to refine abstractions. The evolution loop is a stratified CEGAR: the failure layer localizes the repair, and the shared-trajectory check separates genuine from spurious witnesses. The key difference is that the layer contracts are persistent interfaces, so a repair at one layer reuses the proofs at unchanged boundaries.

Truth maintenance and provenance. de Kleer's assumption-based TMS, McAllester's truth maintenance, and provenance semirings model support and retraction. The decision ledger borrows the idea of explicit support and reopening, but it does not perform general nonmonotonic inference. Selection is explicit and support is strictly earlier-layer, so the warrant is a forward recurrence. The simplification is deliberate: the paper needs impact analysis and reopening, not a general TMS.

Persistent data structures. Driscoll et al. established path copying and persistence. The version model uses an immutable snapshot and content-addressed evidence; the cutoff theorem is about key locality, and a persistent store is one efficient implementation.

Agent software engineering. Recent work on repository-level coding agents (SWE-bench and related benchmarks) and on self-refinement and reflection shows that agents can iterate on code and tests. The layer calculus addresses a different point in the loop: it constrains what an agent may accept as a completed step. It is complementary to better code generation, better test generation, and better retrieval; it supplies the missing intermediate behavioral obligations.

11. Limitations and Open Problems

Section titled “11. Limitations and Open Problems”

Connector admissibility. The soundness theorem is only as strong as the connector’s operational soundness and reduction preservation. The theorem is conditional on validity conditions 5 and 6 (executable realization and profile admissibility); those conditions are part of layer validity, not background assumptions. Rich concurrency — weak memory, Byzantine faults, real-time deadlines — needs richer profiles and correspondingly richer admissibility proofs. The paper’s trace kernel is the base case, not the final word.

Liveness and profiles. The worked example’s safety claim is discharged by the trace kernel; its liveness claim is conditional on the stated fairness and enabling premises and is not derived from the safety proof. A liveness profile needs its own operational composition law and progress-preservation witness, not a relabeling of the safety argument.

Intent correctness. The theorem guarantees that the executable system refines the admitted root intents. It does not guarantee that the root intents are the right requirements. Root authority is an organizational decision; the calculus makes it explicit and auditable, but it cannot make it correct.

Evidence soundness. The gate trusts the evidence checker. A buggy model checker, an inadequate model, or a mis-specified substrate contract can pass a wrong system. The trusted base must be stated and, where possible, minimized. The mechanization target makes the algebraic part small; the operational part remains a hypothesis.

Coverage of the decision ledger. The gate can require a decision for every interface and contract choice, but it cannot force an agent to record every implicit choice inside a leaf implementation. The ledger’s coverage is the declared obligations and interfaces. A leaf is a trust boundary; its contract is what the parent knows.

Test weakening. The gate blocks weakening of protected intents. It does not prevent an agent from writing a weak test for a property that was never formalized. The remedy is to formalize the property as an intent; the framework makes that remedy visible, not automatic.

Scale. The inspection view of a large layer can still be large. The calculus bounds the change region by the dependency cone up to proved unchanged boundaries, and it bounds the size of a local proof, but a root-obligation change can reach the whole system, and it does not bound the number of cells or the cost of a single model-checking query. Tool support, proof caching, and model-checking engineering remain necessary.

Empirical validation. The claim that the protocol reduces agent error rates is a hypothesis. The paper’s contribution is the calculus and the gate; a controlled study with arm-independent ground truth is the natural next step.

Layer choice. The depth LL and the placement of contracts are design decisions. A poor decomposition can make the connector large or the contracts weak. The calculus provides the correctness condition; the architecture remains an engineering choice, guided by the capacity and factoring remarks.

---

12. Conclusion

Section titled “12. Conclusion”

Software development by agents fails when the gap between a requirement and code is filled by unverified invention. The layer calculus closes the gap with explicit intermediate objects: active intents with support, decisions with rationale and reopening conditions, state machines with contracts, and evidence for every behavioral obligation. One relation — behavior inclusion — carries satisfaction, decomposition, refinement, and realization. Stratification makes support a forward recurrence, makes sibling verification a substitution lemma, and makes version evolution reuse every proof at an unchanged interface. The soundness theorem is short because the system is one chain of inclusions, not three bolted-together worlds. The protocol is the operational reading of that chain: read the layer, fix the interface, verify the machine, decompose into contracts, substitute the child proof, publish atomically. The durable-acknowledgement example shows the chain catching the exact error that motivates the framework: an agent that treats a volatile write as durable, and a gate that refuses to let it pass.

Appendix A. Proof Details

Section titled “Appendix A. Proof Details”

B.1 Theorem 1, expanded

Section titled “B.1 Theorem 1, expanded”

The executable sub-development rooted at a cell $v \in V_k$v ∈ Vk is defined by structural recursion on the refinement tree:

- if $k = L$k = L, it is $\mathrm{Beh}_{\mathrm{exec}}(\ell) = \mathrm{beh}(P_\ell)$Behexec(ℓ) = beh(Pℓ) with its declared semantics;
- if $k < L$k < L, it is $\mathrm{Beh}_{\mathrm{exec}}(v)$Behexec(v), the behavior of the concrete wiring C_vCv applied to the executable sub-developments of the children $c \in \mathrm{ch}(v)$c ∈ ch(v).

The induction hypothesis gives, for each child cc, $B_c = \mathrm{Beh}_{\mathrm{exec}}(c) \subseteq \mathrm{beh}(M_c)$Bc = Behexec(c) ⊆ beh(Mc). Condition 5 gives $\mathrm{Beh}_{\mathrm{exec}}(v) \subseteq K_v(\{\alpha_c(B_c)\}_c)$Behexec(v) ⊆ Kv({αc(Bc)}c). The induction hypothesis and monotonicity give $K_v(\{\alpha_c(B_c)\}_c) \subseteq K_v(\{\alpha_c(\mathrm{beh}(M_c))\}_c)$ Kv({αc(Bc)}c) ⊆ Kv({αc(beh(Mc))}c). The inter-layer operator gives $\alpha_c(\mathrm{beh}(M_c)) \subseteq \mathrm{beh}(B_{v,c})$αc(beh(Mc)) ⊆ beh(Bv,c), and monotonicity gives $K_v(\{\alpha_c(\mathrm{beh}(M_c))\}_c) \subseteq K_v(\{\mathrm{beh}(B_{v,c})\}_c)$Kv({αc(beh(Mc))}c) ⊆ Kv({beh(Bv,c)}c). The decomposition obligation gives $K_v(\{\mathrm{beh}(B_{v,c})\}_c) \subseteq \mathrm{beh}(M_v)$Kv({beh(Bv,c)}c) ⊆ beh(Mv). Transitivity gives $\mathrm{Beh}_{\mathrm{exec}}(v) \subseteq \mathrm{beh}(M_v)$Behexec(v) ⊆ beh(Mv). The satisfaction evidence gives $\mathrm{beh}(M_v) \subseteq [\![c]\!]$beh(Mv) ⊆ ⟦c⟧ for every active intent assigned to vv. Condition 6 supplies initialization, input receptiveness, and (where claimed) progress. The root case $k = 0$k = 0 composes the chain to $\Phi_0(v_0) = \bigcap_{c \in A_0(v_0)} [\![c]\!]$Φ0(v0) = ∩c∈A0(v0)⟦c⟧.

B.2 Theorem 3, expanded

Section titled “B.2 Theorem 3, expanded”

The key definitions are functions of their argument tuples. The parent’s wiring-adequacy key lists C_vCv, K_vKv, α_vαv, $(\alpha_c)_c$(αc)c, Γ_vΓv, and $(\Gamma_c)_c$(Γc)c; it is universally quantified over admissible child behaviors and does not list the child implementations. A change to a child interface map changes the parent’s wiring-adequacy key and requires the applicable parent evidence to be re-established. The parent’s satisfaction key for an intent c'c′ lists M_vMv, $[\![c']\!]$⟦c′⟧, and Γ_vΓv; none of these changes. The parent’s decomposition key lists W_vWv, the contracts $B_{v,c}$Bv,c, M_vMv, α_vαv, and Γ_vΓv; none changes. A sibling’s refinement key lists the sibling machine and its own contract; neither changes. An edit to a leaf program changes `k_Impl`; if the leaf machine and its incoming contract are fixed, `k_Ref` and the parent decomposition key are unchanged. A changed child machine changes its refinement key and is re-established. The parent operational instance is re-derived by applying the unchanged universal wiring certificate; no ancestor solver is re-run. Equal keys denote equal propositions; the stored evidence is a proof of the proposition, so substitution preserves soundness. The persistent store’s path copying is an implementation of the same substitution, not a separate proof rule.

B.3 Lemma 3, expanded

Section titled “B.3 Lemma 3, expanded”

Let $\rho = s_0 \xrightarrow{\sigma_1} s_1 \xrightarrow{\sigma_2} \cdots \xrightarrow{\sigma_n} s_n$ρ = s0 σ1

---

s1 σ2

⋯ σn

---

sn be a child execution. Initiation relates s_0s0 to an initial contract state t_0t0. Inductively, $s_i$si is in I_cIc and related to $t_i$ti. For step $i + 1$ i + 1: if $\lambda_c(\sigma_{i+1}) = \epsilon$λc(σi+1) = ϵ, the relation keeps $t_{i+1} = t_i$ti+1 = ti; otherwise the step-simulation condition supplies t_{i+1}ti+1 with the matching contract action. At the end, the sequence of non-silent contract actions is exactly $\lambda_c(\rho)$λc(ρ), so the abstract trace is in $\mathrm{tr}(B_{v,c})$

tr(Bv,c). Input receptiveness ensures the abstract trace is not vacuous at open inputs; silent convergence ensures liveness profiles do not hide infinite internal stuttering.

Appendix B. Notation

Section titled “Appendix B. Notation”

Symbol	Meaning
O_XOX	observation space at interface XX
$\mathrm{beh}(M)$beh(M)	behavior set of machine MM
$\mathrm{Beh}_{\mathrm{exec}}(v)$Behexec(v)	executable behavior of the sub-development rooted at vv
C_vCv	concrete wiring implementation at vv
Q_cQc	an admissible child behavior used in the wiring-adequacy quantifier
$\preceq$ ⪯, $\preceq_\alpha$ ⪯α	refinement (with abstraction αα)
L_kLk	layer kk
I_k, J_kIk, Jk	intents and support links
D_k, N_k, S_kDk, Nk, Sk	decisions, nogoods, selections
V_kVk	cells at layer kk
M_vMv	machine of cell vv
A_kAk, $A_k(v)$Ak(v)	active intents at layer kk, owned by vv
$\Phi_k(v)$Φk(v)	semantic conjunction of active intents owned by vv
$\chi_k(v)$χk(v)	decomposition or realization record
W_vWv	wiring skeleton
K_vKv	connector transformer
$B_{v,c}$Bv,c	contract from vv to child cc
α_cαc	child abstraction map from child observations to the contract interface
E_kEk	evidence map
κκ	obligation key; κ_{Wire}κWire is the wiring-adequacy key over $(C_v, K_v, \alpha_v, (\alpha_c)_c, \Gamma_v, (\Gamma_c)_c)$(Cv, Kv, αv, (αc)c, Γv, (Γc)c)
G_tGt	version at time tt
$\mathrm{stale}(x)$stale(x)	decisions depending on xx
$\Rrightarrow$ ⇛	inter-layer refinement operator

References

Section titled “References”

1. J.-R. Abrial. *The B-Book: Assigning Programs to Meanings.* Cambridge University Press, 1996.
2. R.-J. Back and J. von Wright. *Refinement Calculus: A Systematic Introduction.* Springer, 1998.
3. M. Abadi and L. Lamport. Conjoining Specifications. *ACM Transactions on Programming Languages and Systems*, 17(3):507—534, 1995.
4. M. Abadi and L. Lamport. The Existence of Refinement Mappings. *Theoretical Computer Science*, 82(2):253—284, 1991.
5. L. de Alfaro and T. A. Henzinger. Interface Automata. In *Proc. ESEC/FSE*, pages 109—120, 2001.

6. E. M. Clarke, O. Grumberg, S. Jha, Y. Lu, and H. Veith. Counterexample-Guided Abstraction Refinement for Symbolic Model Checking. *Journal of the ACM*, 50(5):752–794, 2003.
7. J. de Kleer. An Assumption-Based TMS. *Artificial Intelligence*, 28(2):127–162, 1986.
8. J. R. Driscoll, N. Sarnak, D. D. Sleator, and R. E. Tarjan. Making Data Structures Persistent. *Journal of Computer and System Sciences*, 38(1):86–124, 1989.
9. T. J. Green, G. Karvounarakis, and V. Tannen. Provenance Semirings. In *Proc. PODS*, pages 31–40, 2007.
10. D. Harel. Statecharts: A Visual Formalism for Complex Systems. *Science of Computer Programming*, 8(3):231–274, 1987.
11. M. P. Herlihy and J. M. Wing. Linearizability: A Correctness Condition for Concurrent Objects. *ACM Transactions on Programming Languages and Systems*, 12(3):463–492, 1990.
12. C. A. R. Hoare. *Communicating Sequential Processes.* Prentice Hall, 1985.
13. C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *Proc. ICLR*, 2024.
14. W. Kunz and H. W. J. Rittel. Issues as Elements of Information Systems. Working Paper 131, University of California, Berkeley, 1970.
15. L. Lamport. *Specifying Systems: The TLA+ Language and Tools for Hardware and Software Engineers.* Addison-Wesley, 2002.
16. R. J. Lipton. Reduction: A Method of Proving Properties of Parallel Programs. *Communications of the ACM*, 18(12):717–721, 1975.
17. A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang, S. Gupta, B. P. Majumder, K. Hermann, S. Welleck, A. Yazdanbakhsh, and P. Clark. Self-Refine: Iterative Refinement with Self-Feedback. In *Proc. NeurIPS*, 2023.
18. D. McAllester. Truth Maintenance. In *Proc. AAAI*, pages 1109–1116, 1990.
19. B. Meyer. *Object-Oriented Software Construction.* Prentice Hall, 2nd edition, 1997.
20. N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. In *Proc. NeurIPS*, 2023.
21. N. P. Suh. *Axiomatic Design: Advances and Applications.* Oxford University Press, 2001.
22. A. van Lamsweerde. Goal-Oriented Requirements Engineering: A Guided Tour. In *Proc. IEEE RE*, pages 249–262, 2001.
23. S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao. ReAct: Synergizing Reasoning and Acting in Language Models. In *Proc. ICLR*, 2023.